

Основы глубинного обучения

Лекция 3

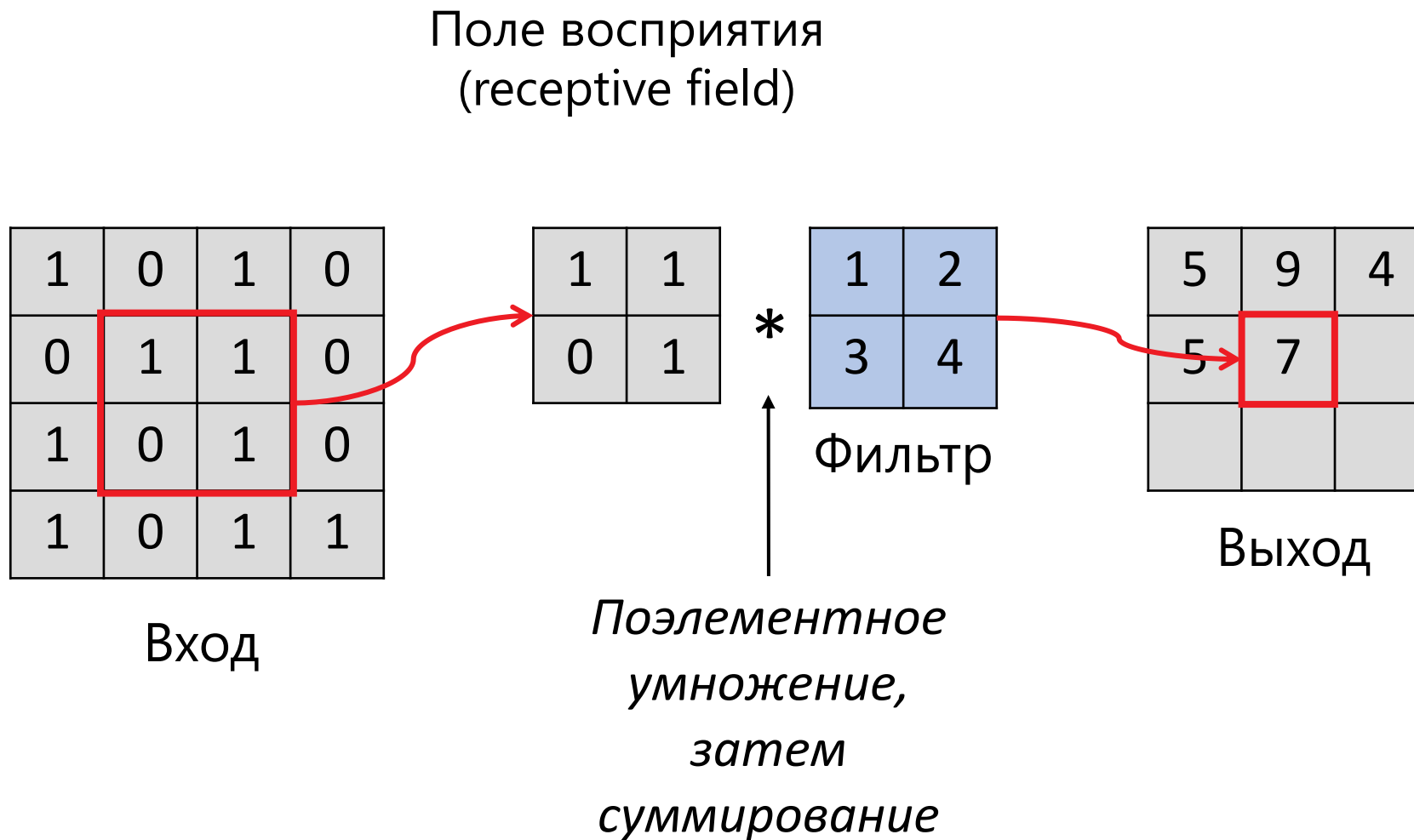
Свёрточные сети

Евгений Соколов

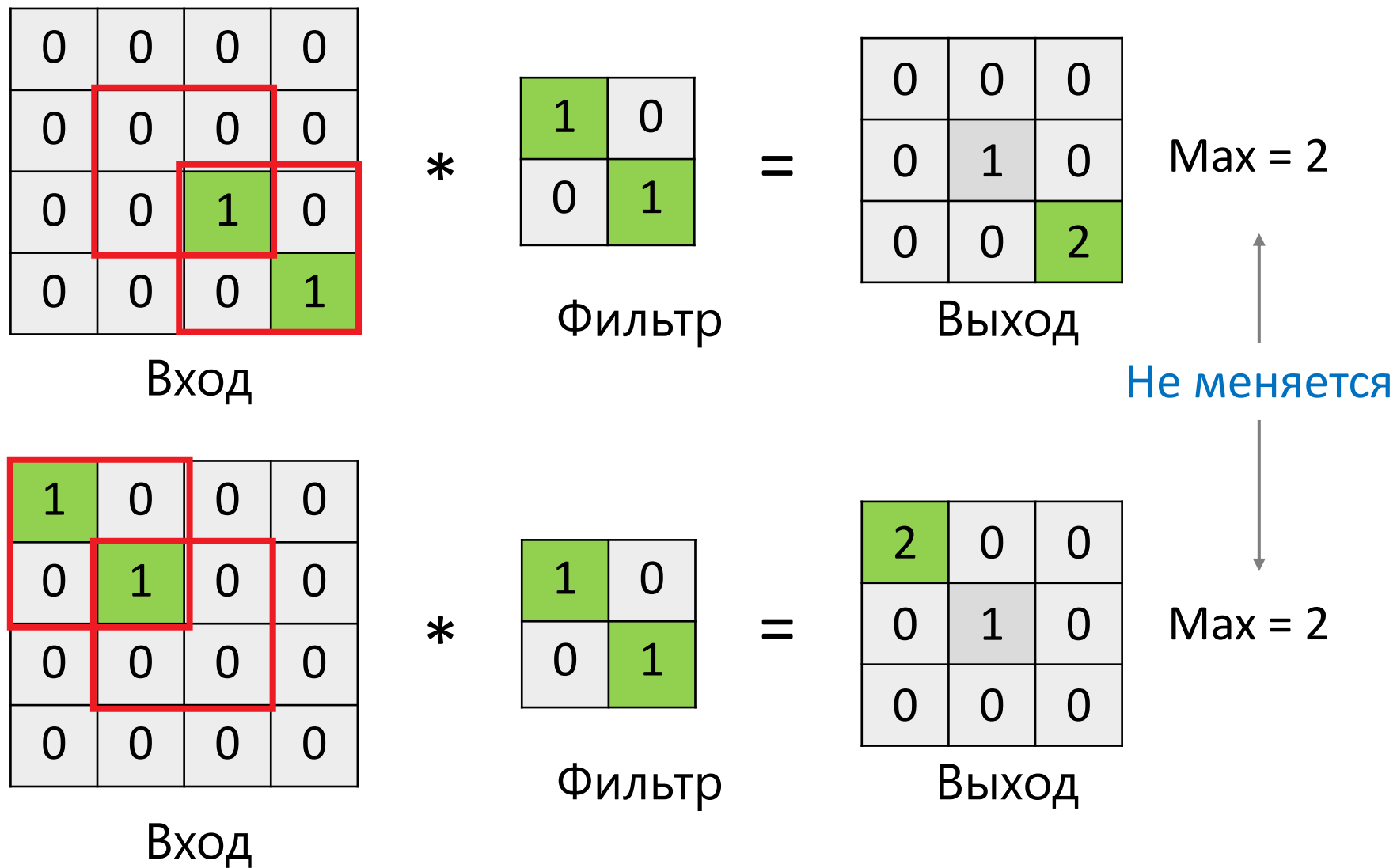
esokolov@hse.ru

НИУ ВШЭ, 2026

Свёртка



Максимум свёртки инвариантен к сдвигам

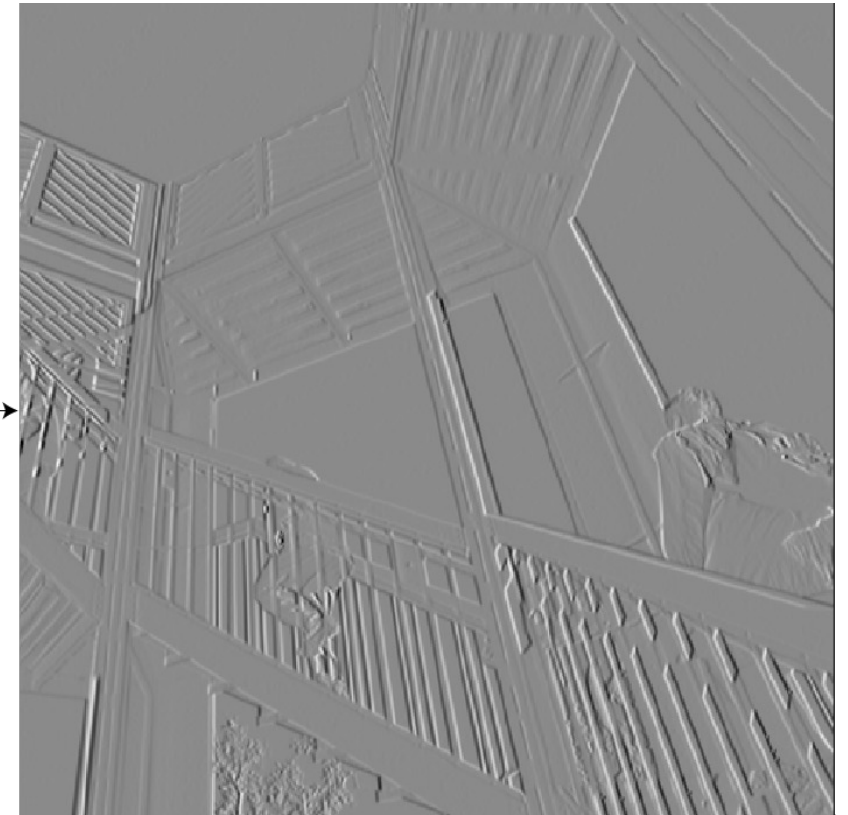


Свёртки в компьютерном зрении



$$\begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix}$$

Horizontal Sobel kernel



Свёртка

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d \left(K(i, j) \text{Im}^{in}(x + i, y + j) \right) + b$$

Свёртка

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d \left(K(i, j) \text{Im}^{in}(x + i, y + j) \right) + b$$

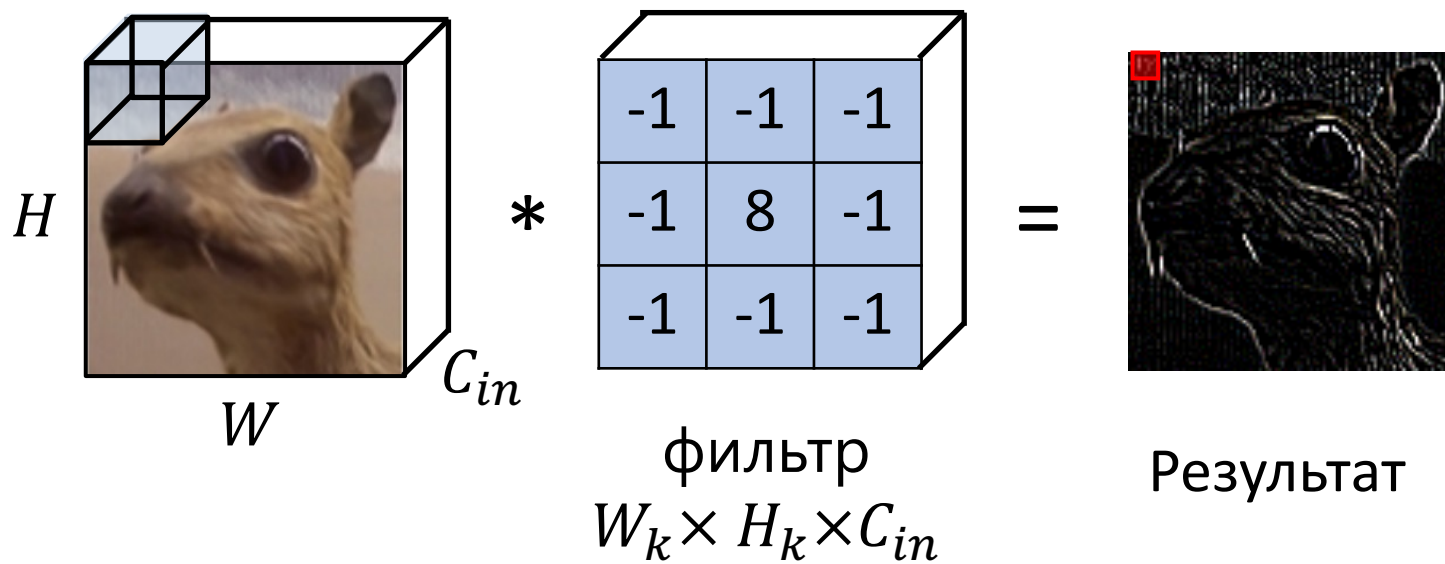
- Пиксель в результирующем изображении зависит только от небольшого участка исходного изображения (local connectivity)
- Веса одни и те же для всех пикселей результирующего изображения (shared weights)

Свёртка

- Обычно исходное изображение цветное!
- Это означает, что в нём несколько каналов (R, G, B)
- Учтём в формуле:

$$\text{Im}^{out}(x, y) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + b$$

Свёртка



Свёртка

- Одна свёртка выделяет конкретный паттерн на изображении
- Нам интересно искать много паттернов
- Сделаем результат трёхмерным:

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + b_t$$

Число параметров

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + b_t$$

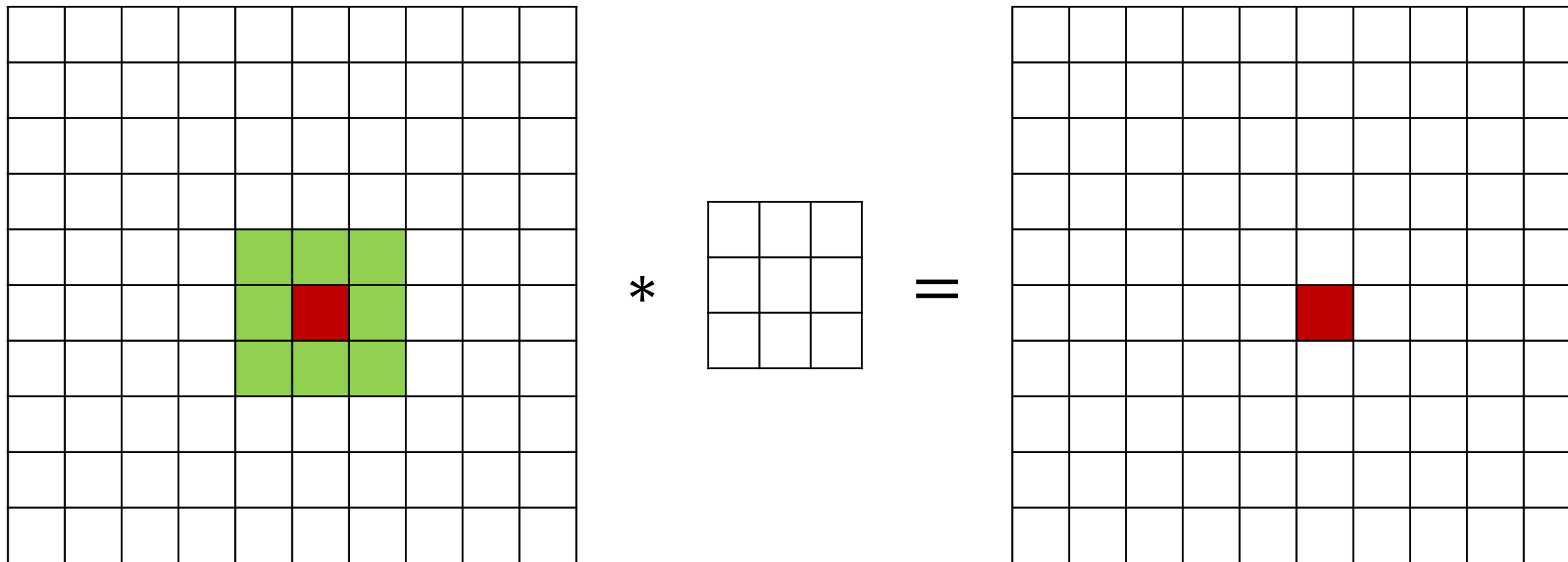
- Обучается только фильтр
- $((2d + 1)^2 * C + 1) * T$ параметров
- Как из этого сделать модель — обсудим позже

Receptive field

Receptive field

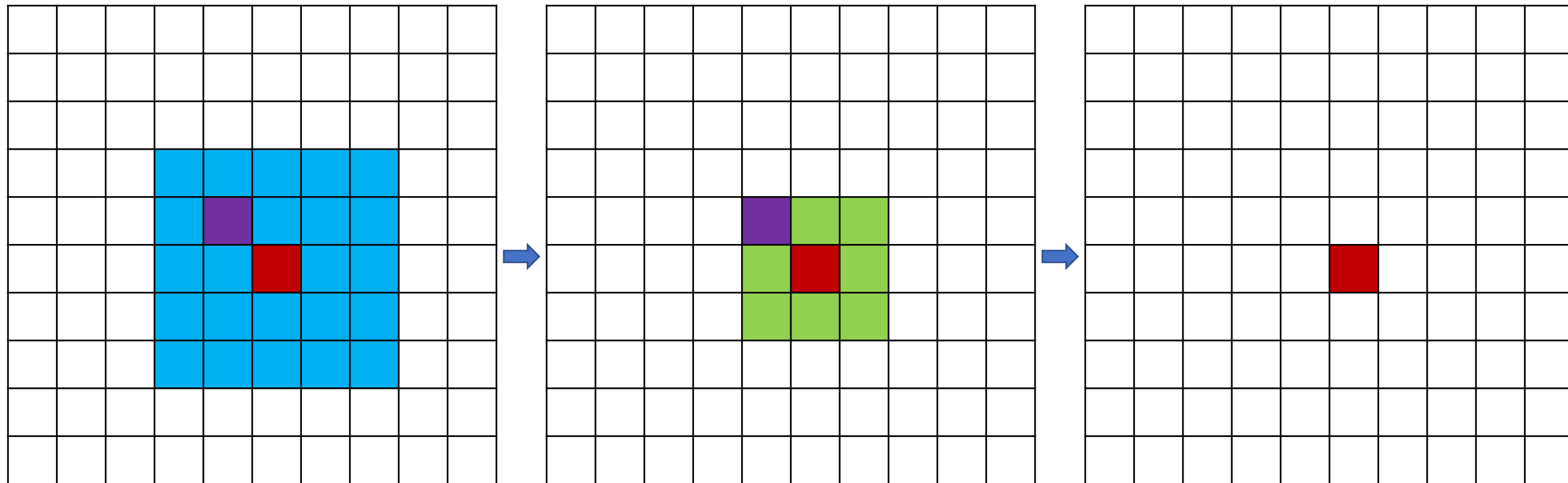
- Возьмём пиксель в итоговом изображении (после свёрточных слоёв)
- От какой части входного изображения зависит значение в этом пикселе?

Receptive field



Поле восприятия: 3 x 3

Receptive field



Поле восприятия: 5 x 5

Receptive field

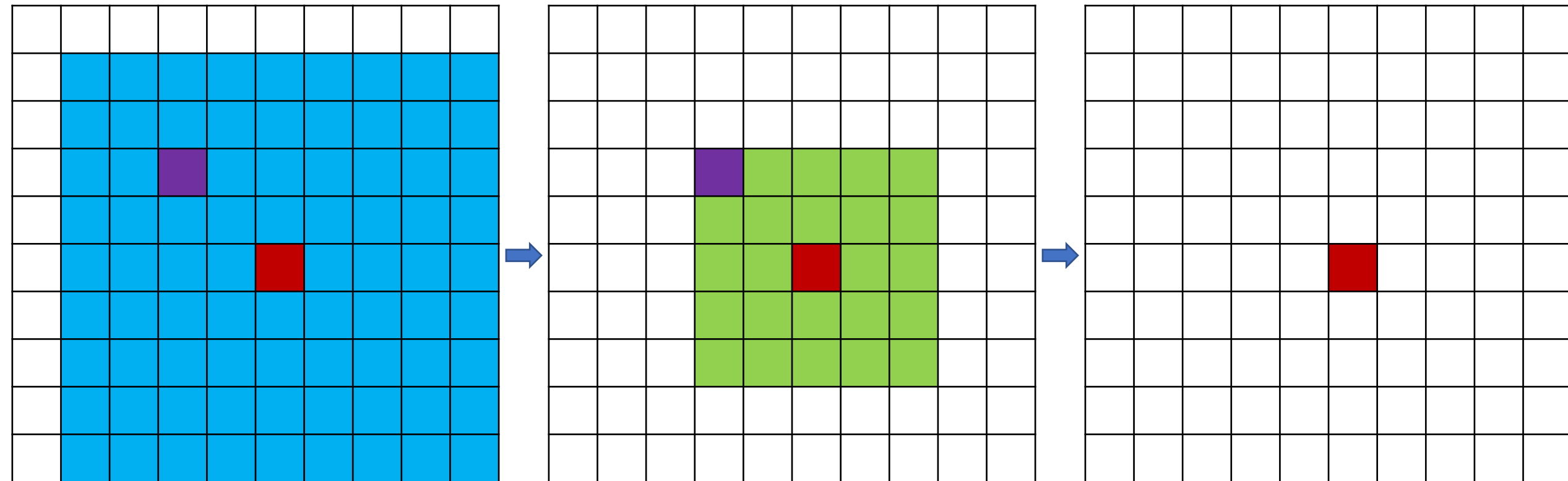
Поле восприятия для свёртки 3 x 3:

- После 1 свёрточного слоя: 3 x 3
- После 2 свёрточных слоёв: 5 x 5
- После 3 свёрточных слоёв: 7 x 7

Receptive field

Поле восприятия для свёртки 5 x 5:

Receptive field



Поле восприятия: 9 x 9

Receptive field

Поле восприятия для свёртки 5 x 5:

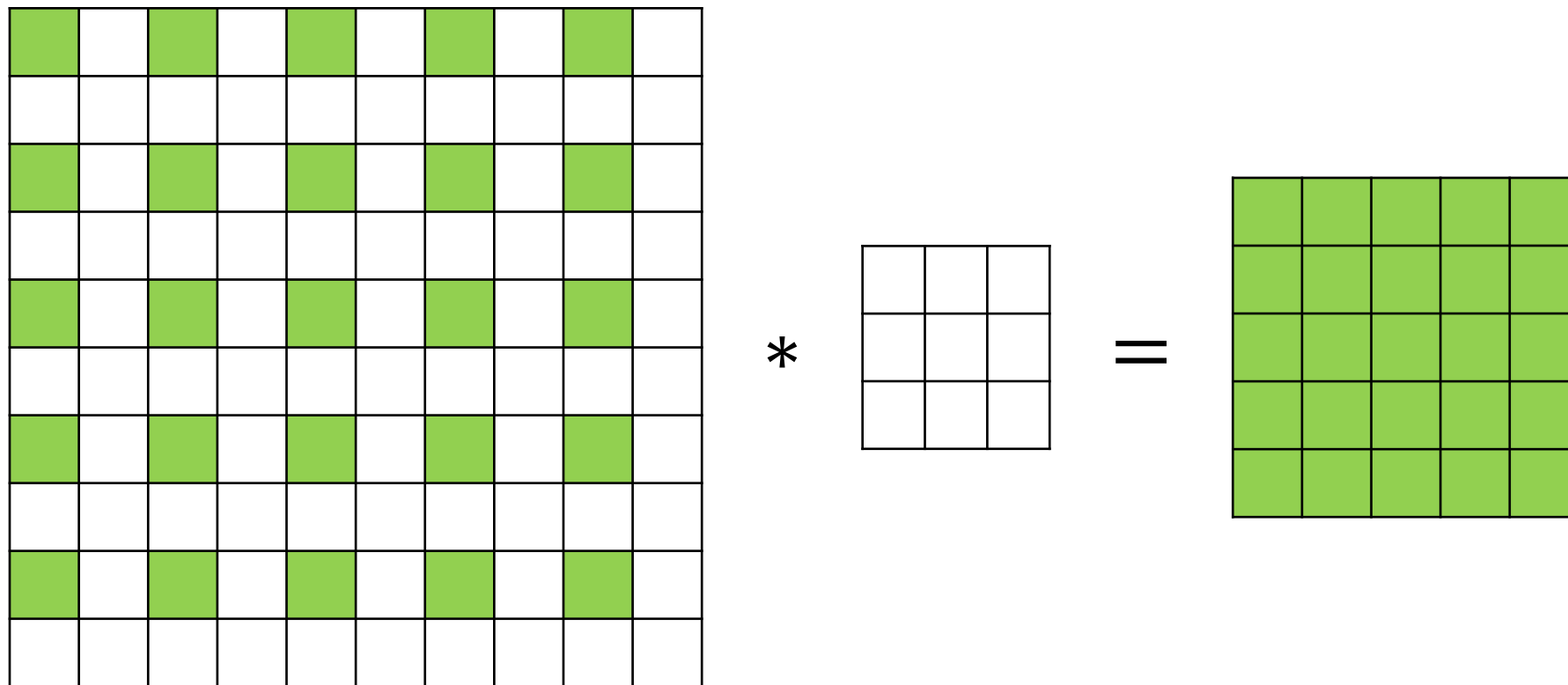
- После 1 свёрточного слоя: 5 x 5
- После 2 свёрточных слоёв: 9 x 9
- После 3 свёрточных слоёв: 13 x 13

Нужно очень много слоёв, если изображение размера 512 x 512

Почему фильтры маленькие

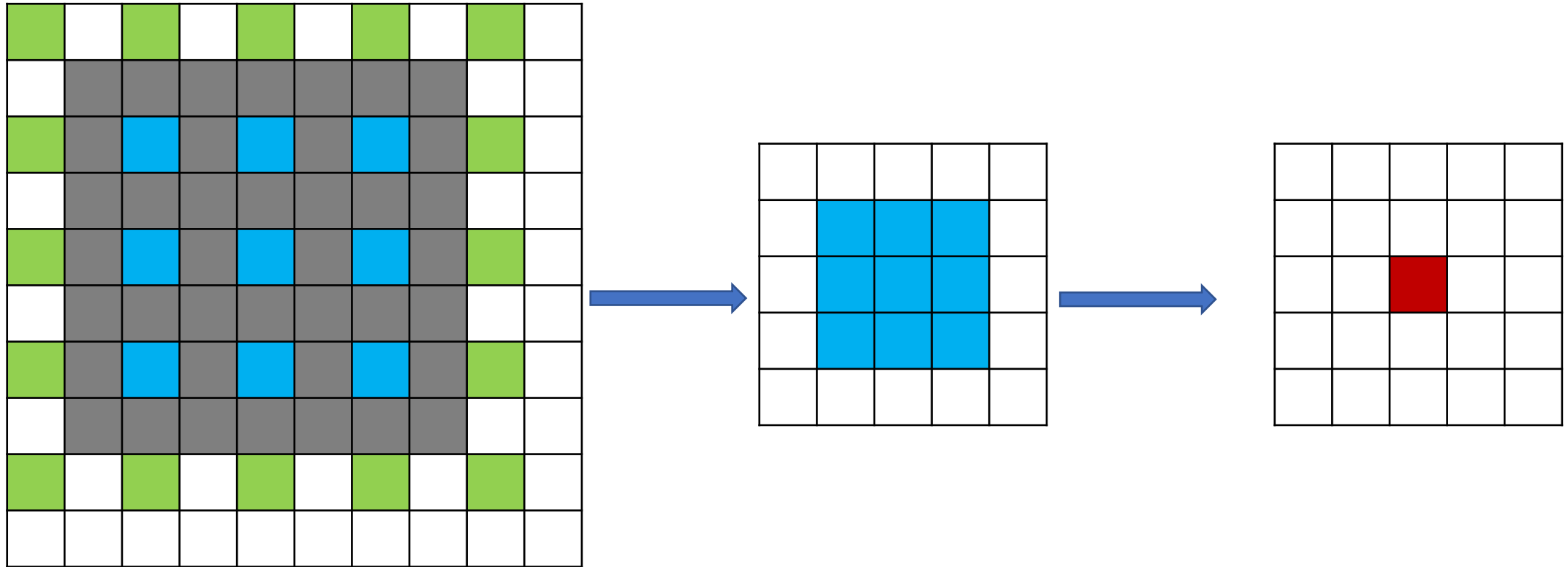
- Три слоя 3×3 дают поле восприятия 7×7
 - как один слой 7×7
- Параметров меньше:
 - один слой 7×7 : 49
 - три слоя 3×3 : $3 \cdot 9 = 27$ — почти вдвое меньше!
- Нелинейностей больше: три вместо одной → модель выразительнее
- Стандартом надолго стали свёртки 3×3 (VGG, 2014)

Свёртки с пропусками (strides)



$$s = 2$$

Свёртки с пропусками (strides)



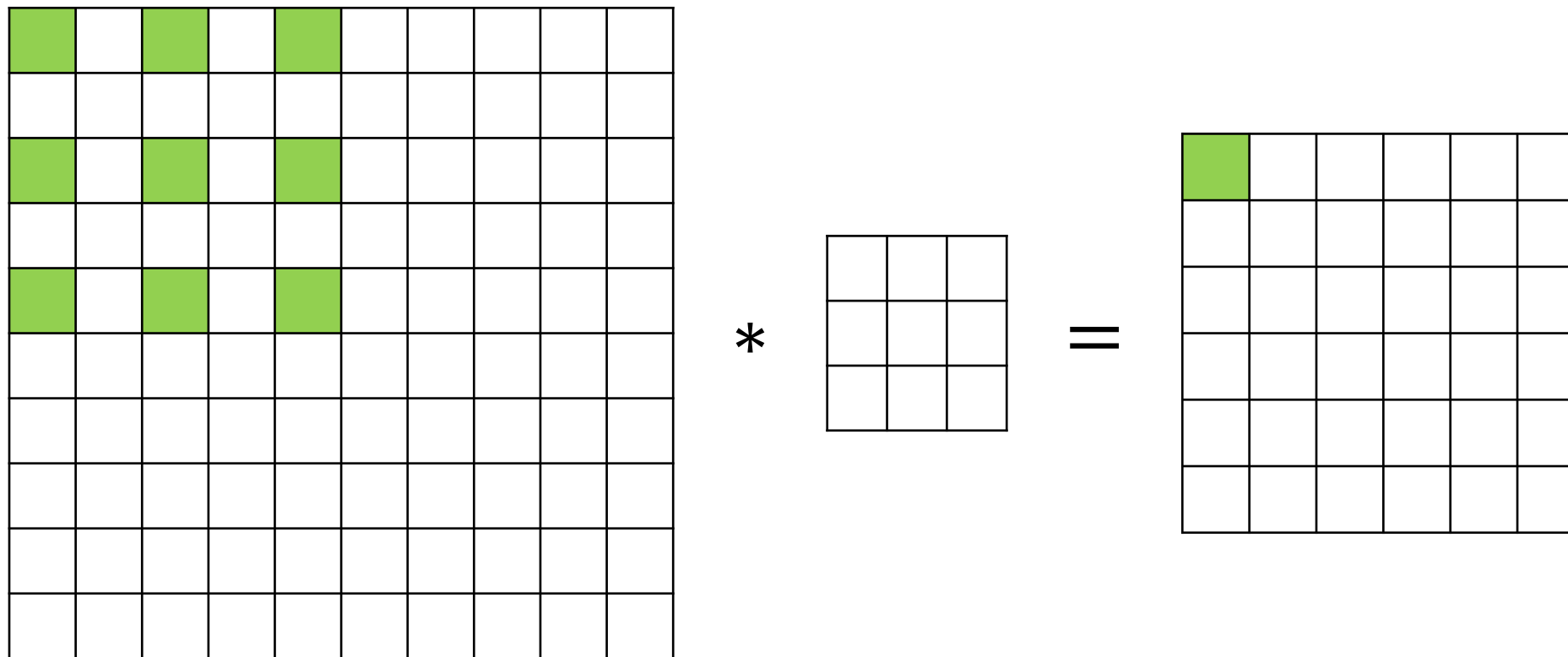
Поле восприятия: 7 x 7

Свёртки с пропусками (strides)

Подробности про подсчёт размера поля:

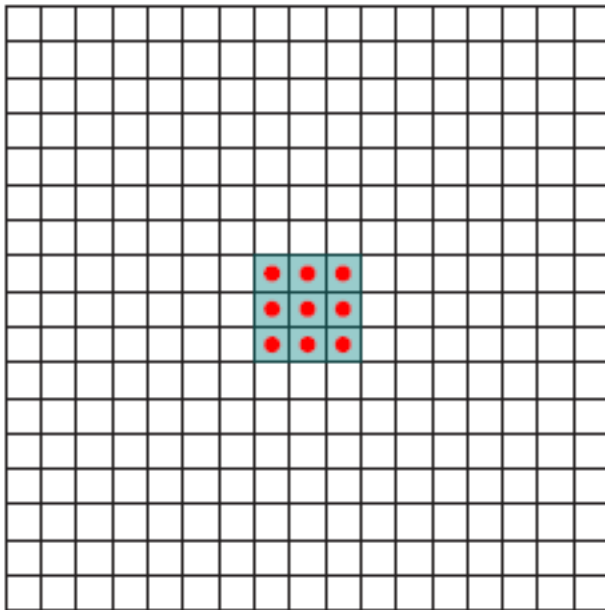
<https://distill.pub/2019/computing-receptive-fields/>

Dilated convolutions («раздутые» свёртки)

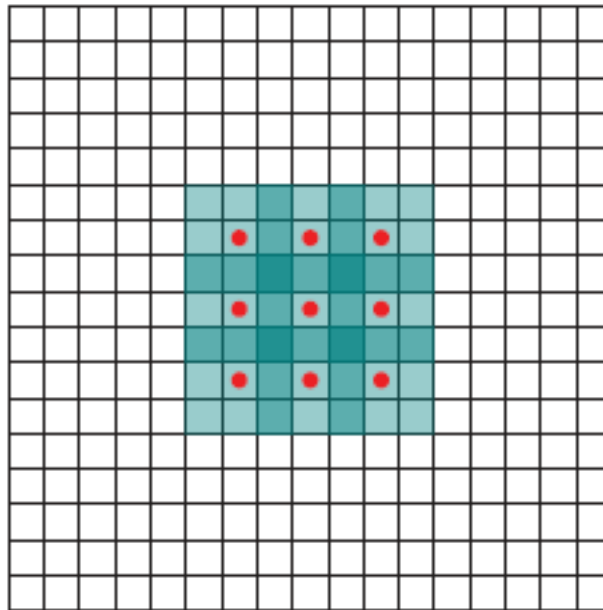


$$l = 2$$

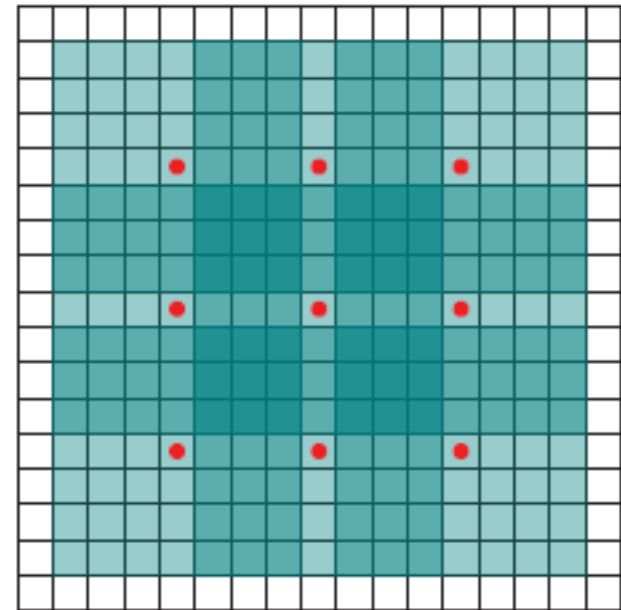
Dilated convolutions («раздутые» свёртки)



$l = 1$



$l = 2$



$l = 4$

Stride vs dilation

- Оба увеличивают поле восприятия, не добавляя параметров
- Stride: уменьшает изображение → дешевле, но теряем разрешение
 - Классификация, эмбединги
- Dilation: размер сохраняется → дороже, зато ответ в каждом пикселе
 - Сегментация, звук (WaveNet)

Pooling

1	0	2	1	0	0
0	1	3	2	1	2



1	3	2

Max-pooling с фильтром 2x2

Pooling

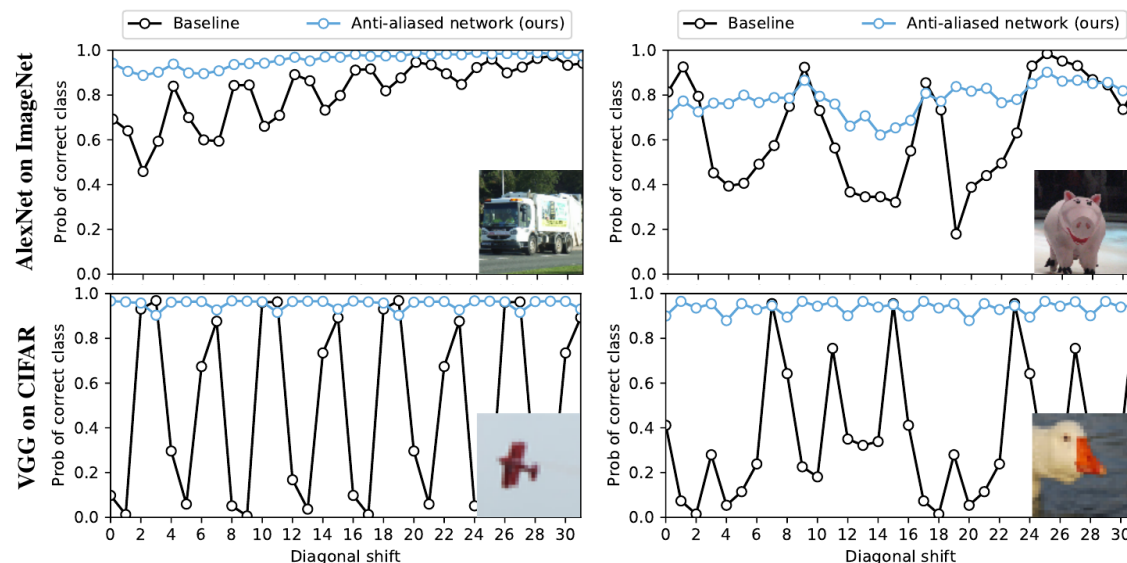
- Разбивает изображение на участки $n \times m$ и считает некоторую статистику в каждом участке (обычно максимум или среднее)
- Существенно сокращает размер изображения (значит, увеличивает поле восприятия следующих слоёв)
- Не имеет параметров
- Каждый канал обрабатывается отдельно

Pooling в PyTorch

- `nn.MaxPool2d(kernel_size)`, `nn.AvgPool2d(kernel_size)`
- Шаг по умолчанию равен размеру окна: `MaxPool2d(2)` уменьшает картинку вдвое
- `nn.AdaptiveAvgPool2d(1)` — одно число на канал при любом размере входа (global average pooling)

Инвариантность к сдвигам — не совсем

- stride и pooling прореживают карту
- из-за этого сдвиг картинки на 1 пиксель может заметно изменить ответ сети
- причина — алиасинг
- можно починить: размыть карту перед прореживанием (BlurPool)

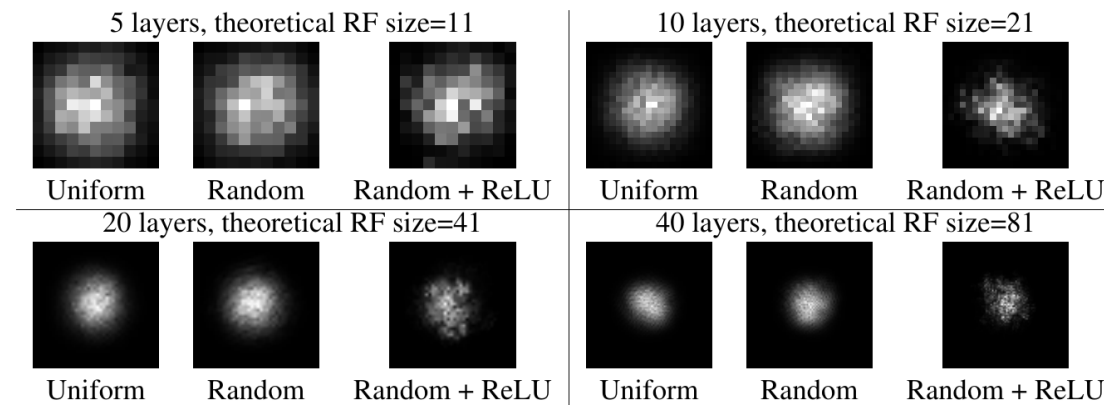


Зачем это всё?

- Важно следить за тем, чтобы последние свёрточные слои имели размер поля восприятия, сравнимый со всей картинкой

Effective receptive field

- Смотрим, как выход в центре зависит от пикселей входа (производная выхода по пикселю)
- Сильнее всего влияет на выход центр, к краям спадает
- Теоретическое поле растёт с числом слоёв n линейно, а эффективное — только как $\sqrt{n}$
- Вывод: поле восприятия должно быть даже больше, чем размер изображения
- После обучения эффективное поле восприятия становится шире



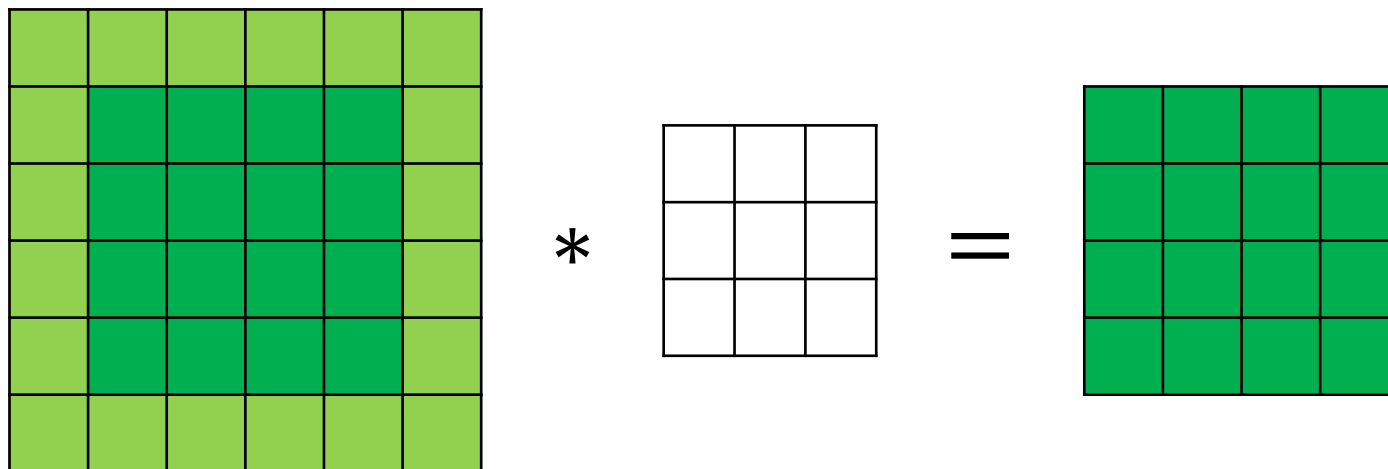
Свёртки 3×3; квадрат — теоретическое поле восприятия, яркость — влияние пикселя на выход

Padding

Свёртки

- Если применять свёртку по формуле, то выходное изображение будет меньше входного

Свёртки



Valid mode

- При честном подсчёте свёрток пиксели на краях не оказывают большого влияния на результат

Не увидим, что фильтр имеет хороший отклик при помещении центра в этот пиксель

0	1				
1	1				

*

0	0	1
0	0	1
1	1	1

Zero padding

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

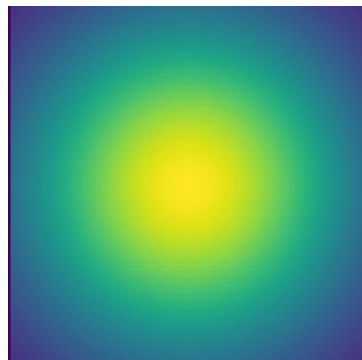
A 6x6 grid of squares, consisting of 6 rows and 6 columns, totaling 36 squares. The grid is used for drawing a net of a cube.

Zero padding

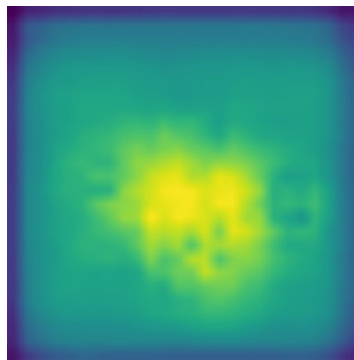
- Добавляем по границам нули так, чтобы посчитанная после этого свёртка в `valid mode` давала изображение такого же размера, как исходное
- Есть риск, что модель научится понимать, где на изображении края — можем потерять инвариантность
- И это происходит!

Zero padding выдаёт положение

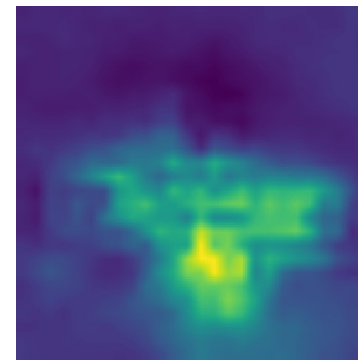
- Берём обученную VGG16, замораживаем
- Добавляем поверх одну свёртку 3×3
- Учим её рисовать одну и ту же карту для любой картинки: светлый центр, тёмные края
- Карта не зависит от содержимого → получится, только если признаки VGG знают координаты
- С паддингом получается (корреляция 0.81), без паддинга — заметно хуже (0.36)



Цель



VGG с паддингом



VGG без паддинга

Reflection padding

[illegible]

*

[illegible]

Reflection padding

- Не получится легко находить края изображения
- Но теперь модель может начать находить зеркальные отражения и подбирать фильтры под них

Replication padding

[illegible]

[illegible]

Replication padding

- Пиксель на границе равен ближайшему пикселю из изображения
- Модель всё ещё может настроиться под паттерны, которые возникают из-за такого паддинга

Circular padding

Строка 1 2 3 4 5 6, паддинг 2:

	слева	строка	справа
zeros	0 0	1 2 3 4 5 6	0 0
reflect	3 2	1 2 3 4 5 6	5 4
replicate	1 1	1 2 3 4 5 6	6 6
circular	5 6	1 2 3 4 5 6	1 2

- Circular: картинка склеена по краям, как тор
- Имеет смысл только для периодических данных: панорамы 360°, глобус и т.д.

Размер выхода

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1$$

k — размер фильтра, p — паддинг, s — шаг

- $k = 3, p = 1, s = 1$: $32 \rightarrow 32$ (размер сохраняется; в общем случае $p = (k - 1)/2$)
- $k = 5, p = 0, s = 1$: $32 \rightarrow 28$ (valid mode)
- $k = 3, p = 1, s = 2$: $32 \rightarrow 16$
- $k = 7, p = 3, s = 2$: $224 \rightarrow 112$ (первый слой ResNet)

Резюме

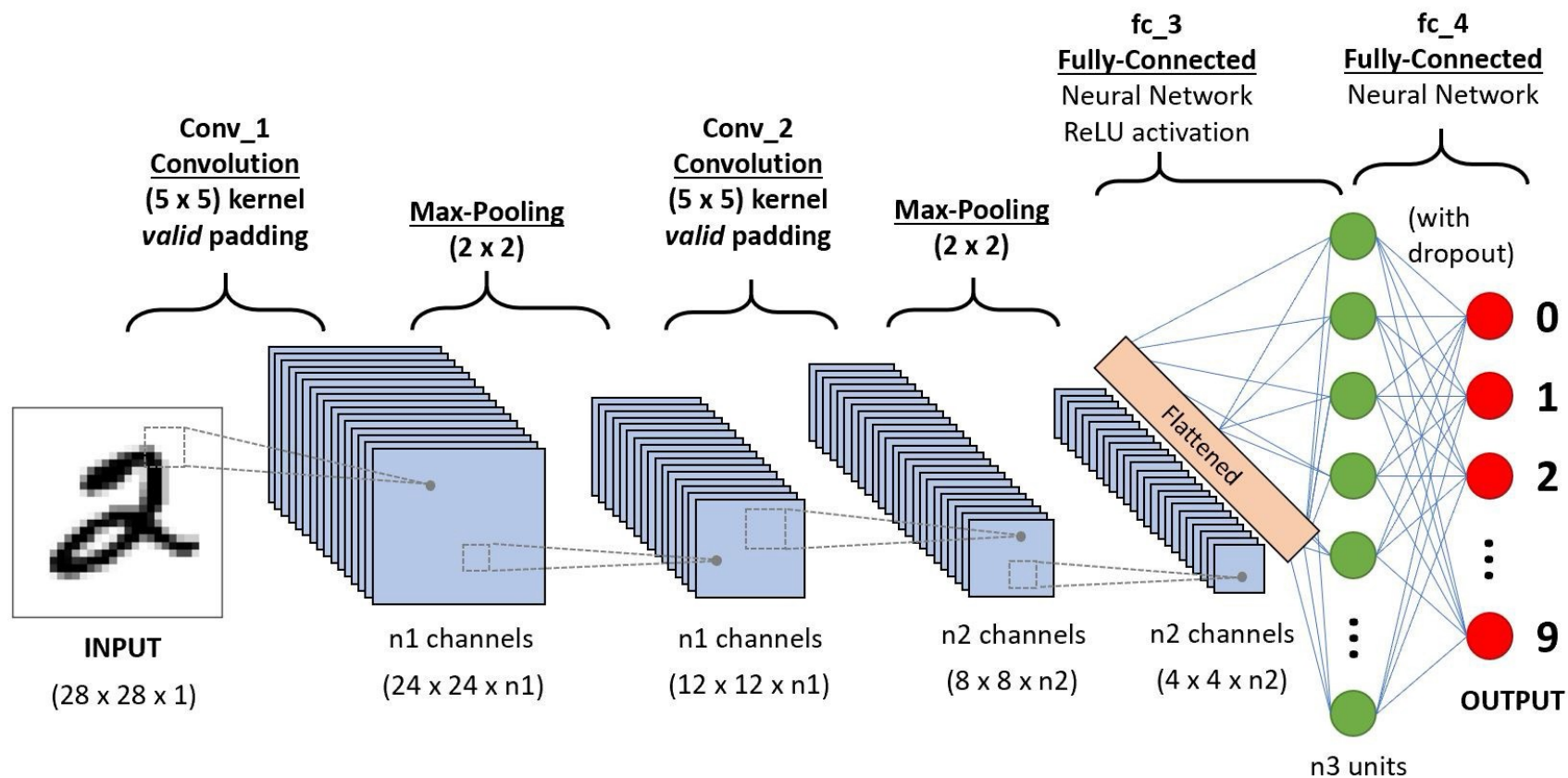
- Паддинг позволяет контролировать размер выходных изображений
- Паддинг позволяет учитывать даже объекты на краях
- Разные типы паддингов допускают разные способы переобучения под края

Структура свёрточных сетей

Свёрточный слой

$$\text{Im}^{out}(x, y, t) = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C \left(K_t(i, j, c) \text{Im}^{in}(x + i, y + j, c) \right) + b_t$$

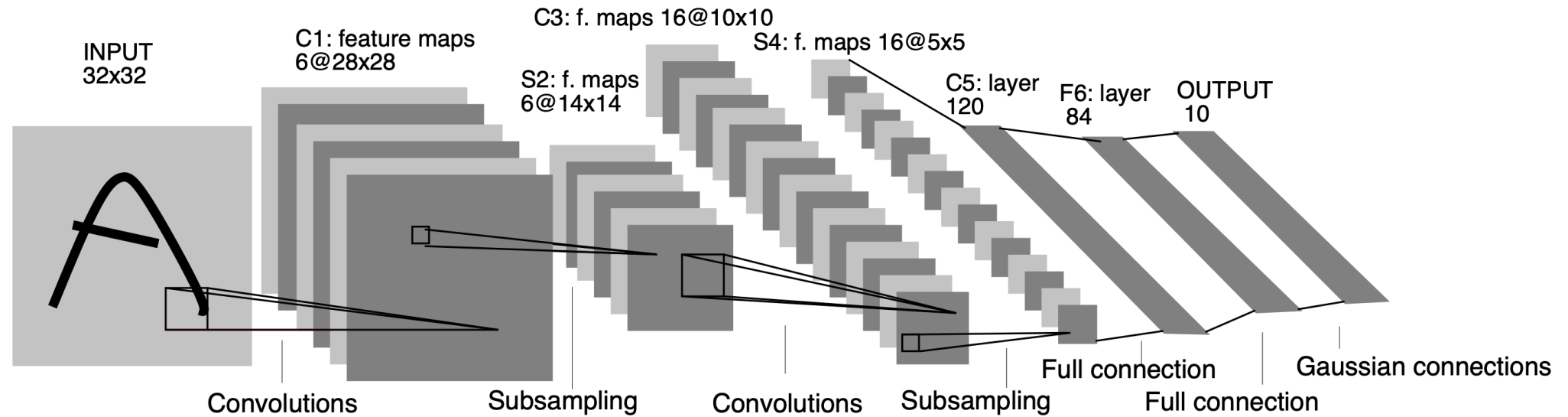
Типичная архитектура



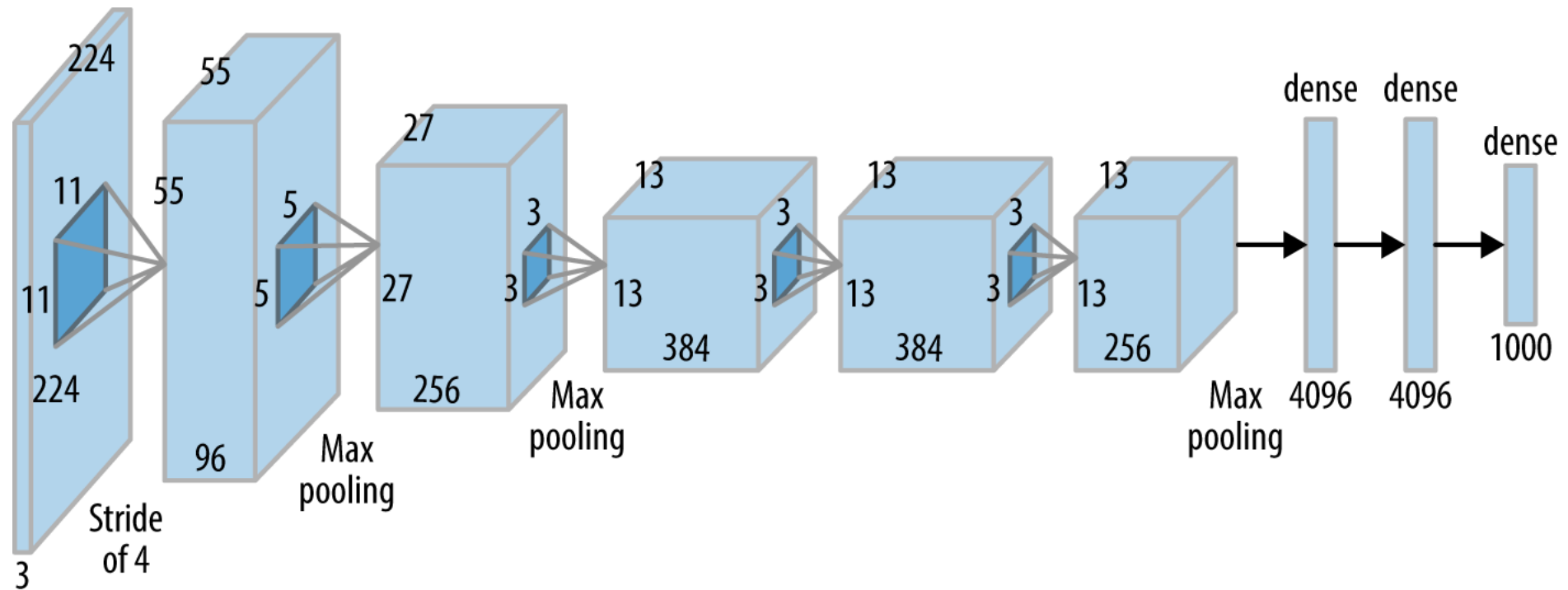
Типичная архитектура

- Последовательное применение комбинаций вида «свёрточный слой → нелинейность → pooling» или «свёрточный слой → нелинейность»
- Выпрямление (flattening) выхода очередного слоя
- Серия полносвязных слоёв
- Так устроены ранние сети (LeNet, AlexNet, VGG). В современных:
 - вместо flatten и серии FC-слоёв — global average pooling и один линейный слой
 - max-pooling остался только в начале сети, дальше свёртки со stride=2

LeNet



AlexNet



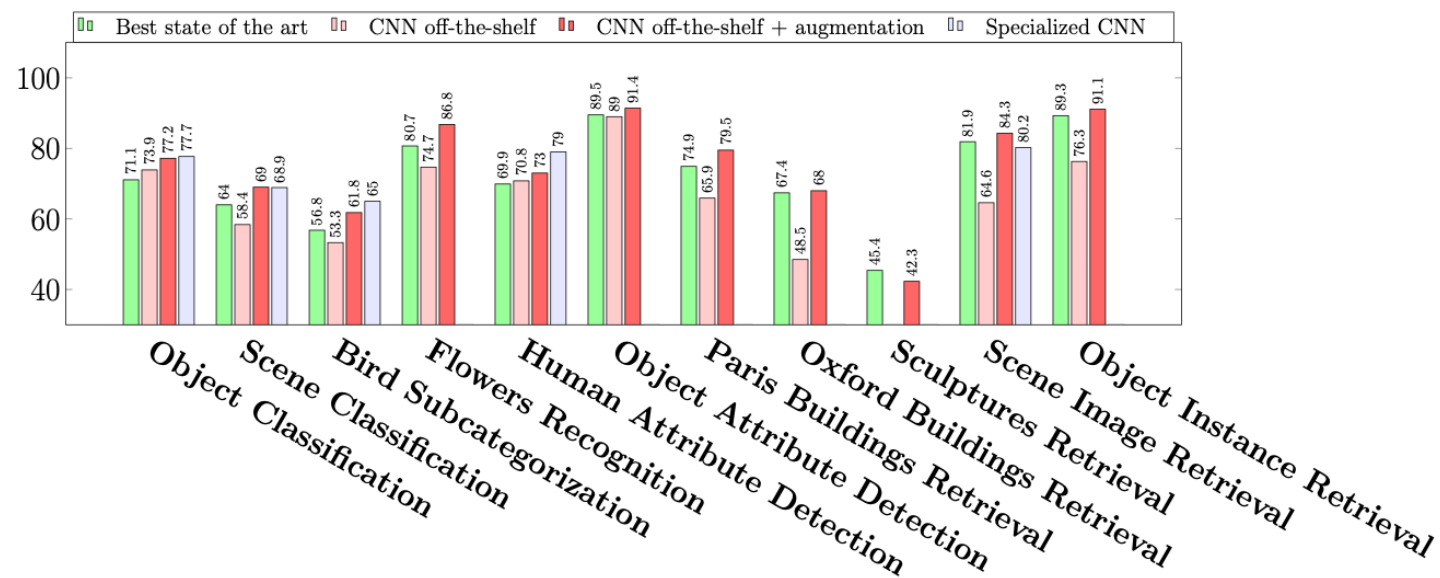
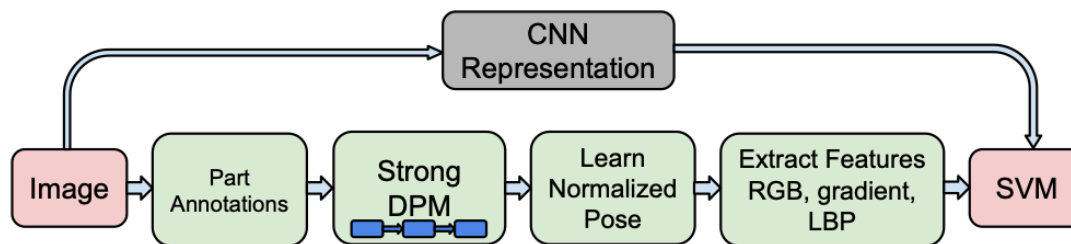
Параметры в AlexNet

- Всего ≈ 61 млн параметров
- Свёрточные слои $\approx 2,3$ млн
- Полносвязные слои $\approx 58,6$ млн (96%!)
 - первый из них: $6 \cdot 6 \cdot 256 = 9216$ входов $\times$ 4096 выходов $\approx 37,7$ млн
- Global average pooling: $256 \times 6 \times 6 \rightarrow 256$ чисел $\rightarrow$ линейный слой
 - ResNet-50: $2048 \rightarrow 1000$, ≈ 2 млн весов из 25,6 млн

Представления с последних слоёв

- Важное наблюдение: выходы последних слоёв являются хорошими признаковыми описаниями изображений
- Полезны во многих задачах
- Например, поиск похожих изображений

Представления с последних слоёв



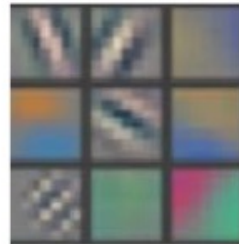
Сейчас: кодировщики изображений

- Сеть учат ради хороших признаков на огромных данных
- CLIP: картинки вместе с подписями, 400 млн пар
 - задача: угадать, какая подпись к какой картинке относится (в батче из N пар найти N правильных среди N^2 сочетаний)
 - признаки картинки и текста попадают в одно пространство → поиск картинок по тексту
- DINOv2 / DINOv3: вообще без разметки
 - задача: по маленькому кусочку картинки предсказать то, что сеть-«учитель» выдаёт по всей картинке
 - DINOv3 — 1,7 млрд изображений, 7 млрд параметров

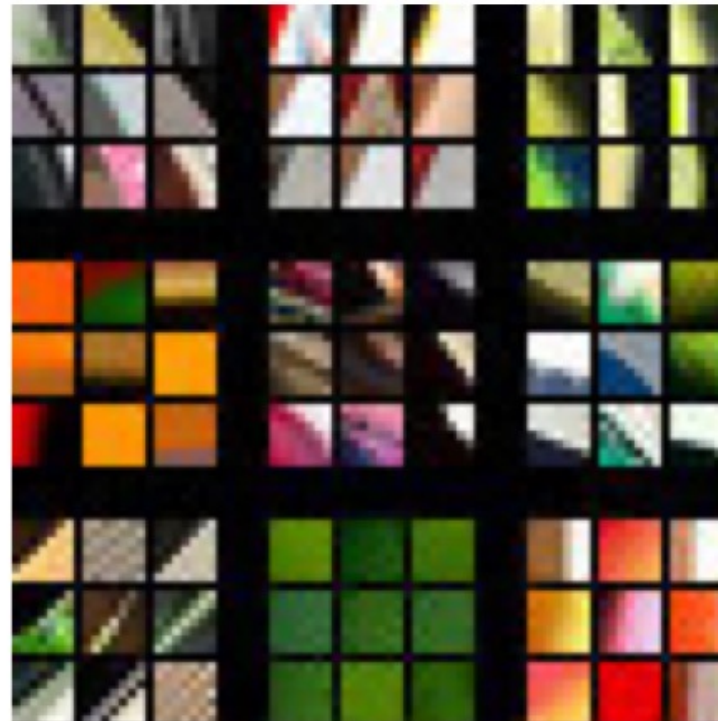
Представления с последних слоёв

- Не интерпретируется (в отличие от классического компьютерного зрения)
- По смыслу — «индикаторы» наличия каких-то паттернов

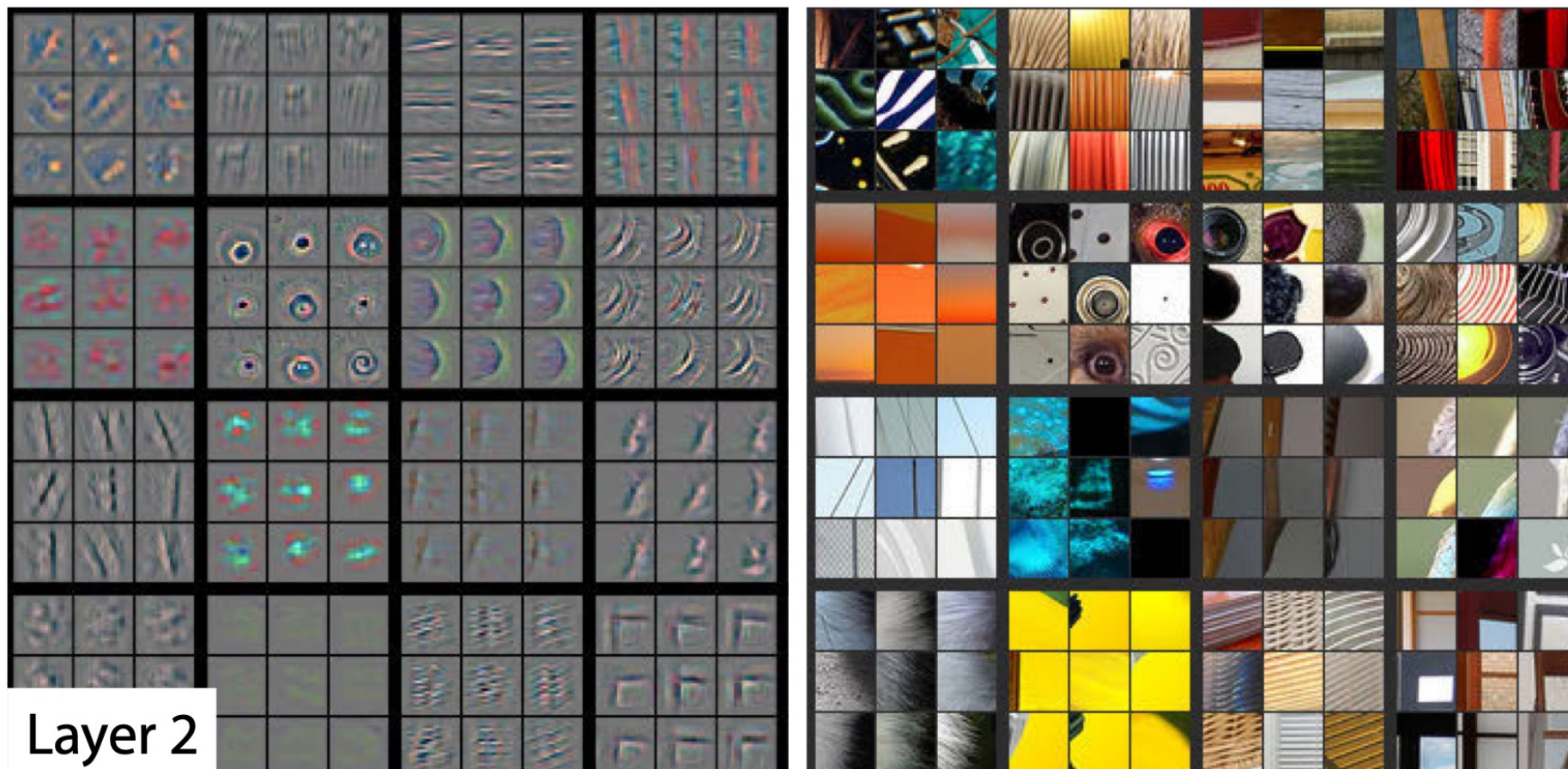
Представления с разных слоёв



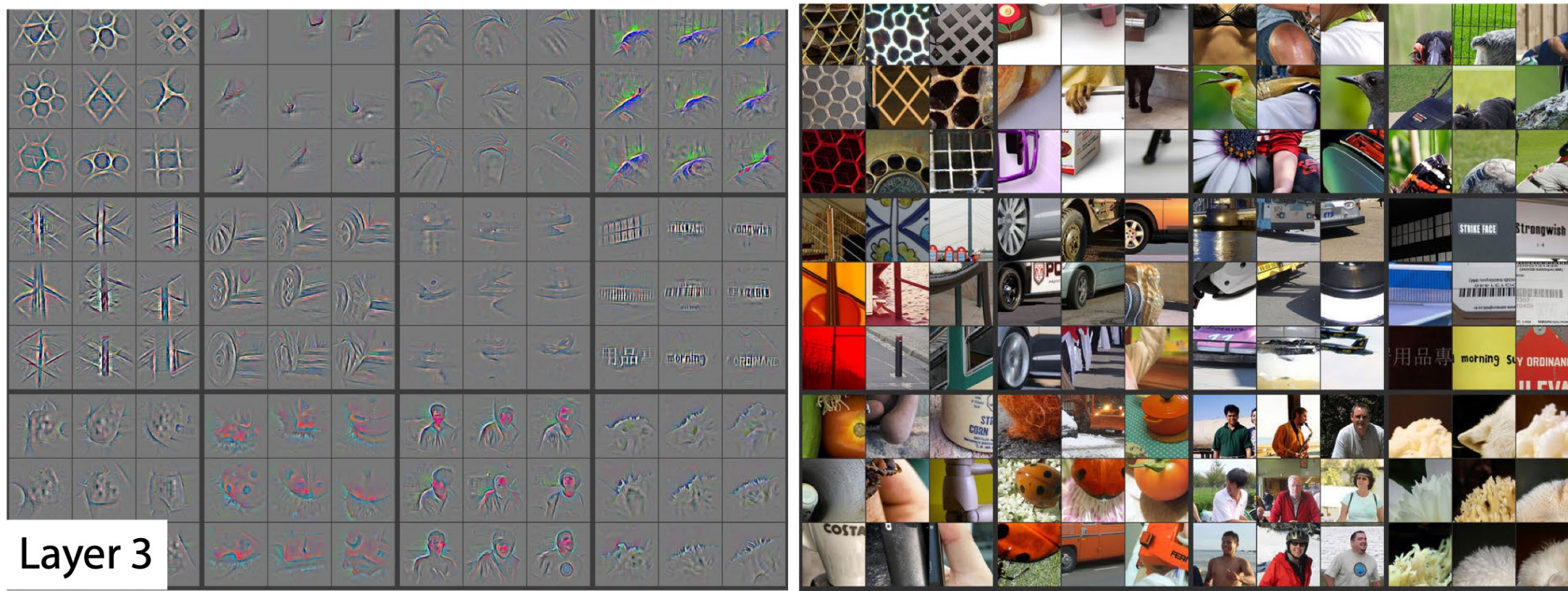
Layer 1



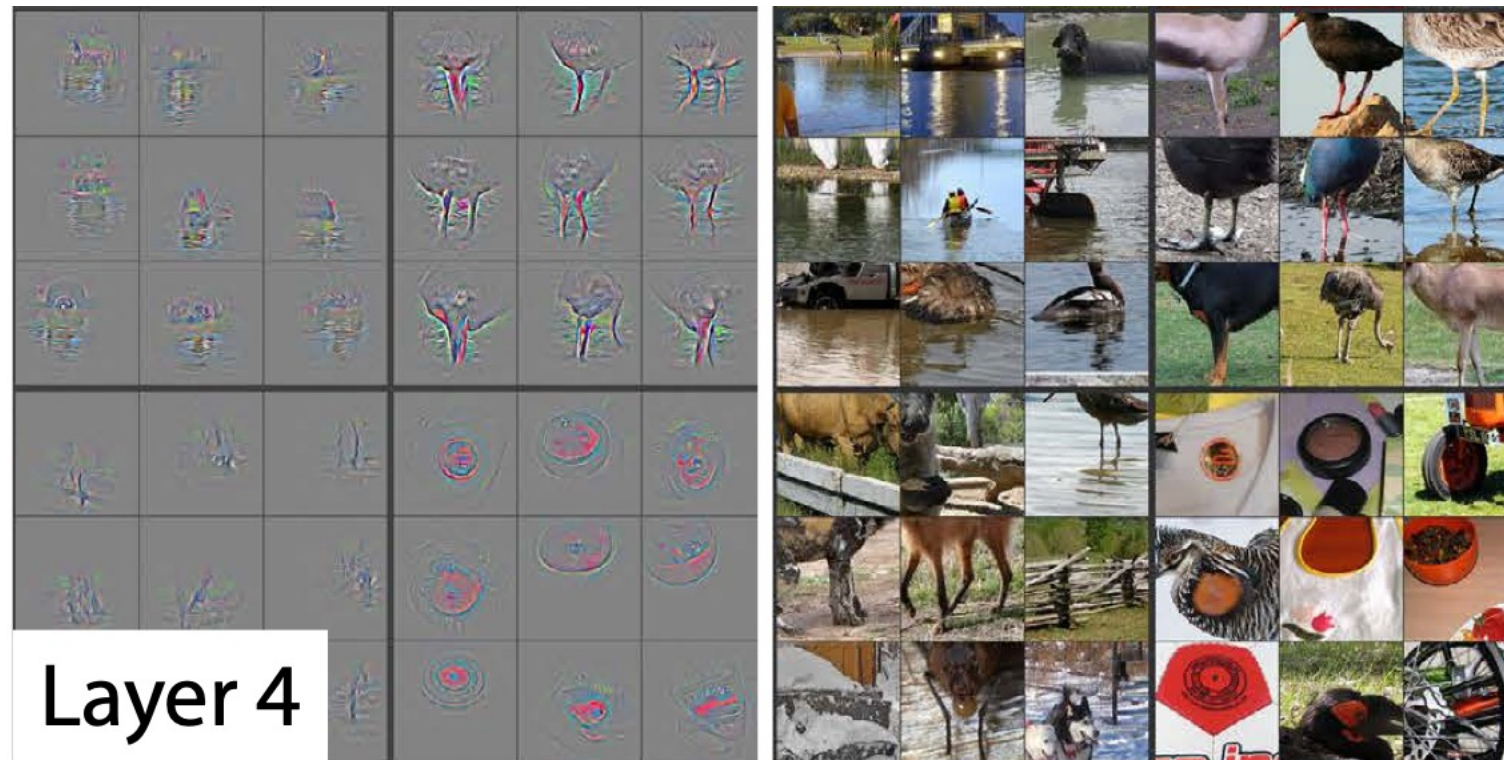
Представления с разных слоёв



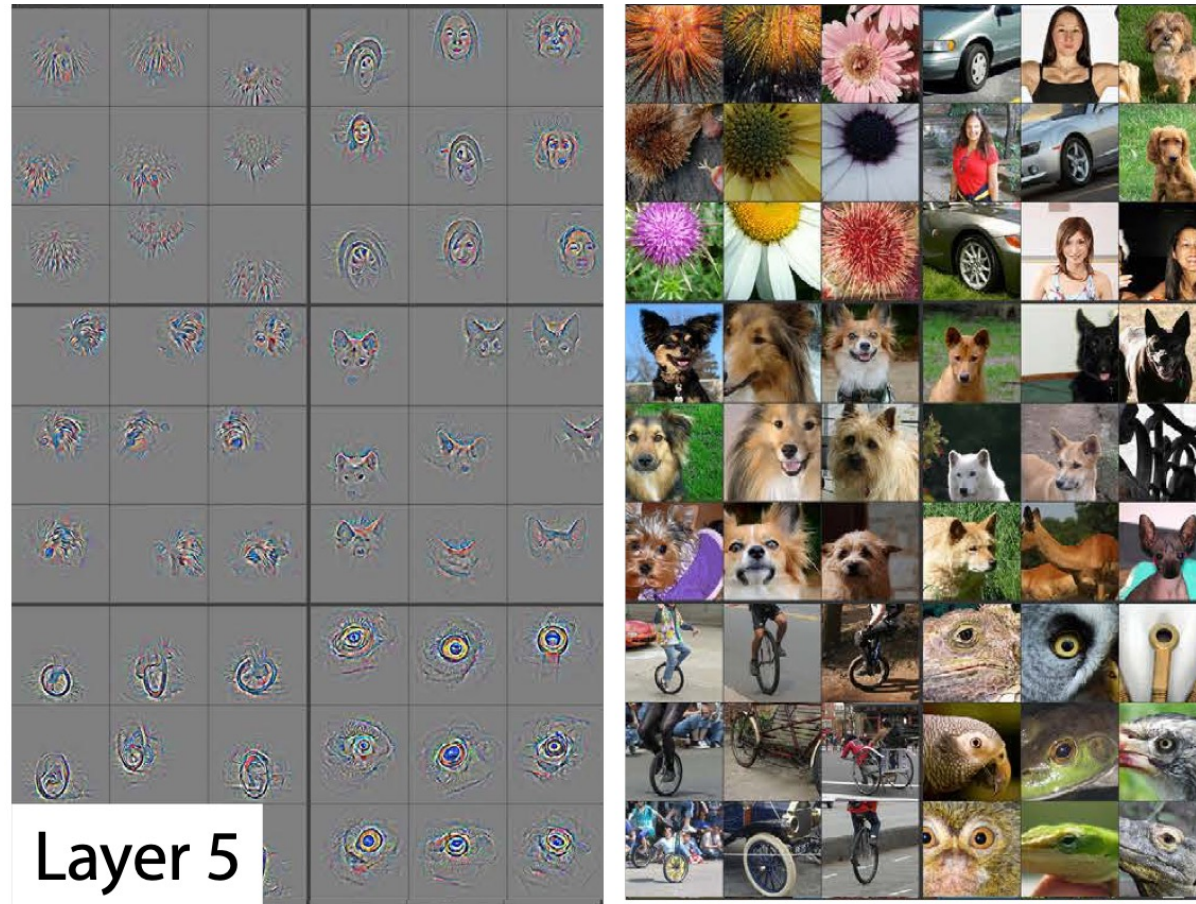
Представления с разных слоёв



Представления с разных слоёв



Представления с разных слоёв



Стохастический градиентный спуск

Градиентный спуск

1. Начальное приближение: w^0

2. Повторять:

$$w^t = w^{t-1} - \eta \nabla Q(w^{t-1})$$

3. Останавливаемся, если ошибка на валидационной выборке перестаёт убывать

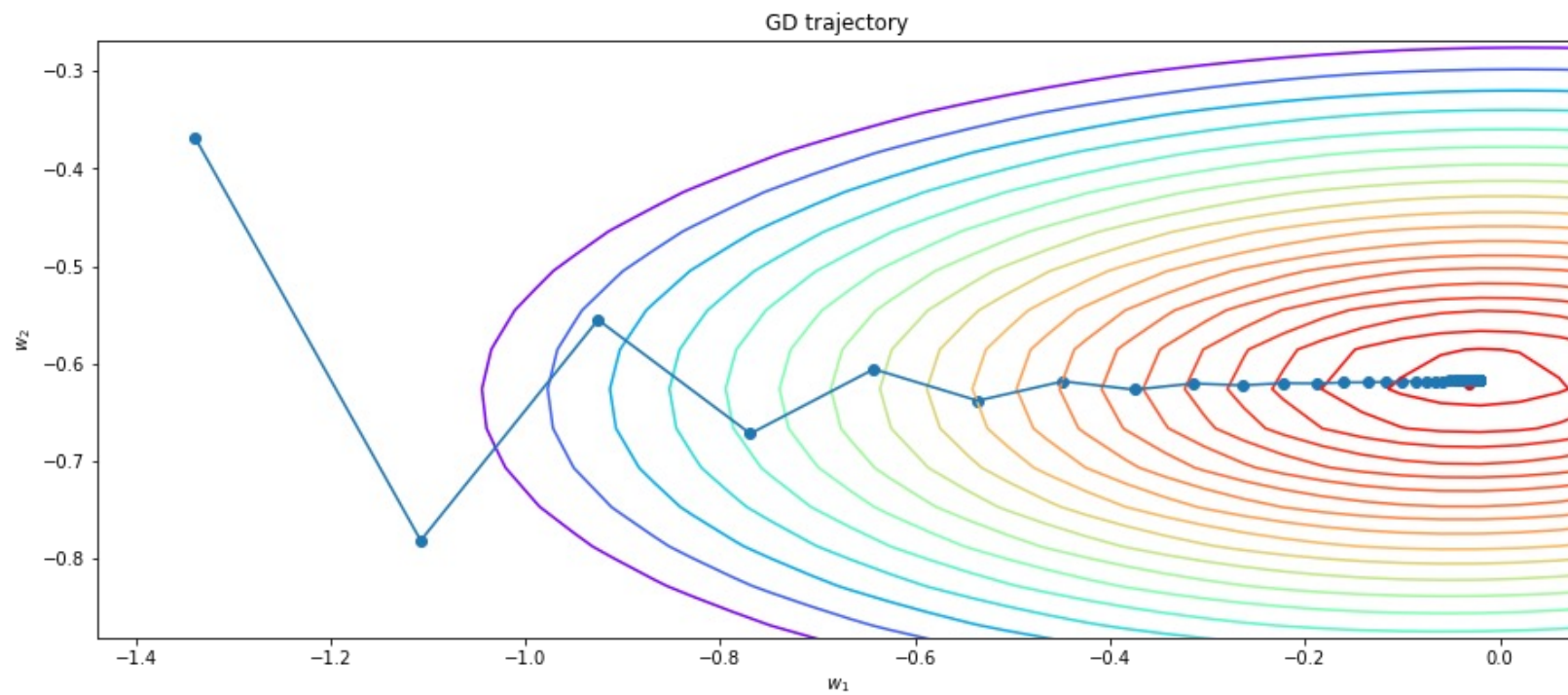
Стохастический градиентный спуск

1. Начальное приближение: w^0
2. Повторять, каждый раз выбирая случайный объект i_t :

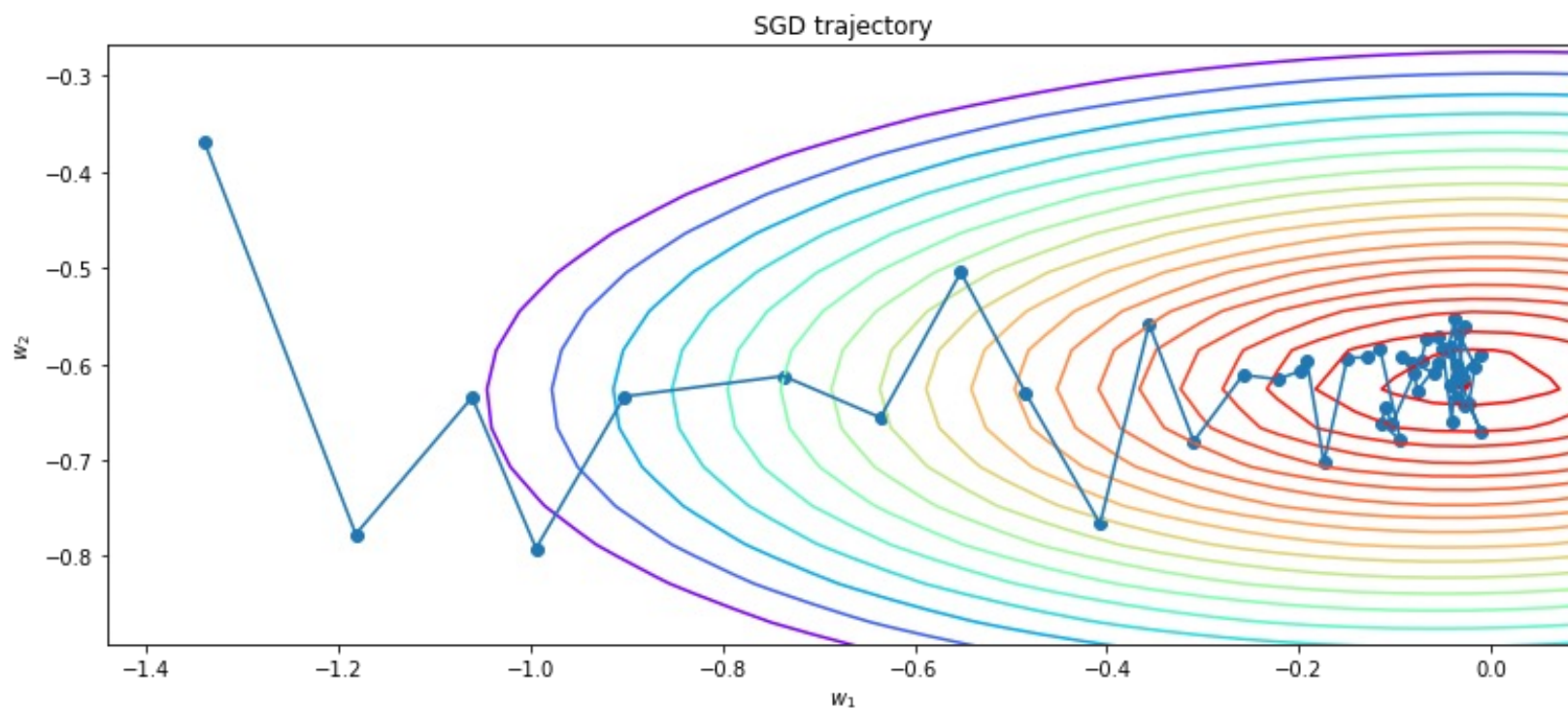
$$w^t = w^{t-1} - \eta \nabla L(y_{i_t}, a(x_{i_t}))$$

3. Останавливаемся, если ошибка на валидационной выборке перестает убывать

Градиентный спуск



Стохастический градиентный спуск



Стохастический градиентный спуск

1. Начальное приближение: w^0
2. Повторять, каждый раз выбирая случайный объект i_t :

$$w^t = w^{t-1} - \eta_t \nabla L(y_{i_t}, a(x_{i_t}))$$

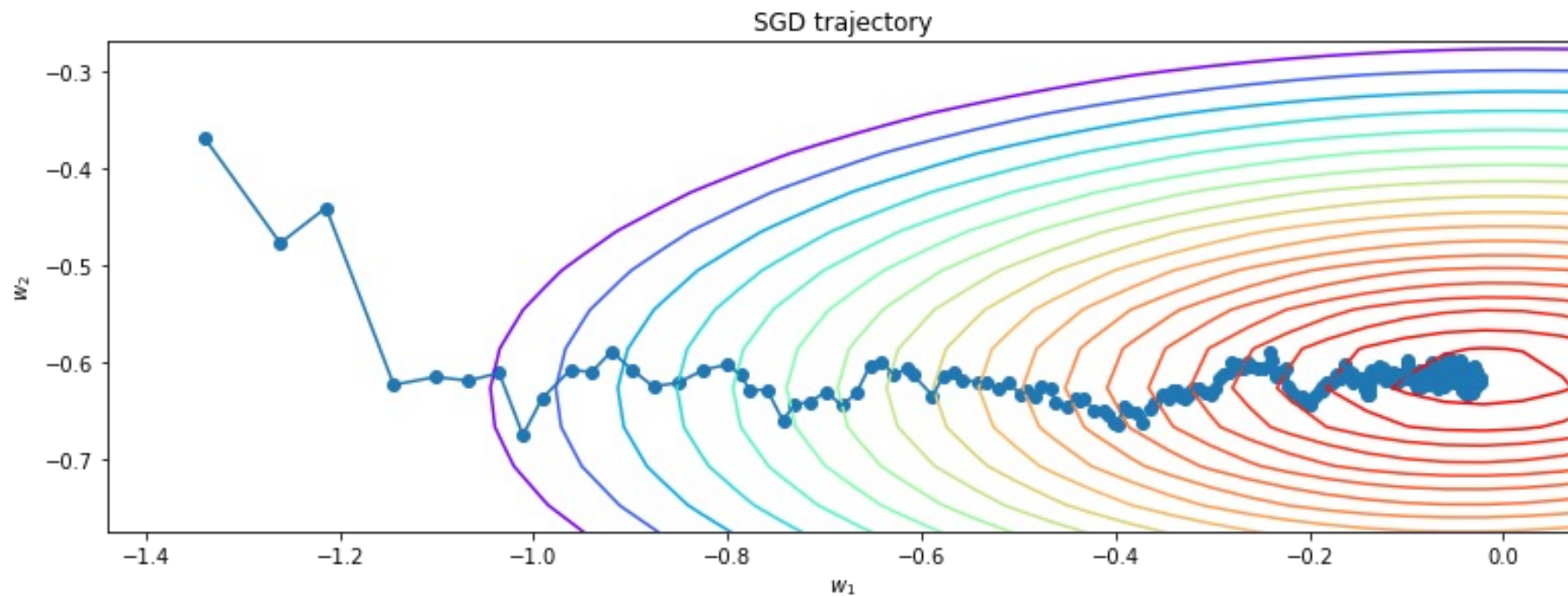
3. Останавливаемся, если ошибка на валидационной выборке перестает убывать

Стохастический градиентный спуск

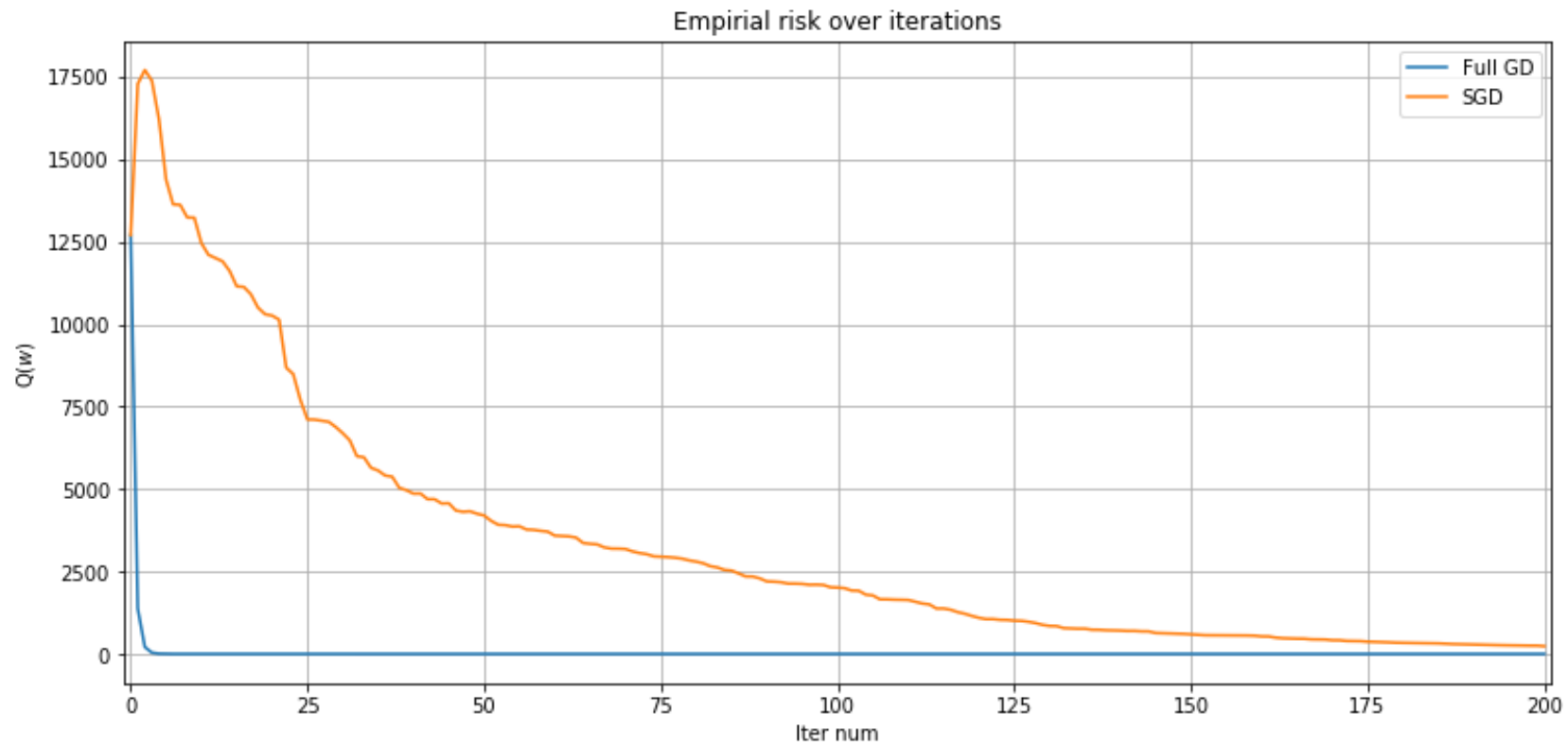
- Оценка по одному объекту **несмещённая**
- То есть в среднем мы идём в правильную сторону
- Даже в точке оптимума оценка по одному объекту вряд ли будет нулевой
- Поэтому важно, чтобы длина шага стремилась к нулю
- Сходимость к глобальному минимуму гарантируется только для выпуклых функций

Стохастический градиентный спуск

$$\eta_t = \frac{0.1}{t^{0.3}}$$



Стохастический градиентный спуск



Mini-batch GD

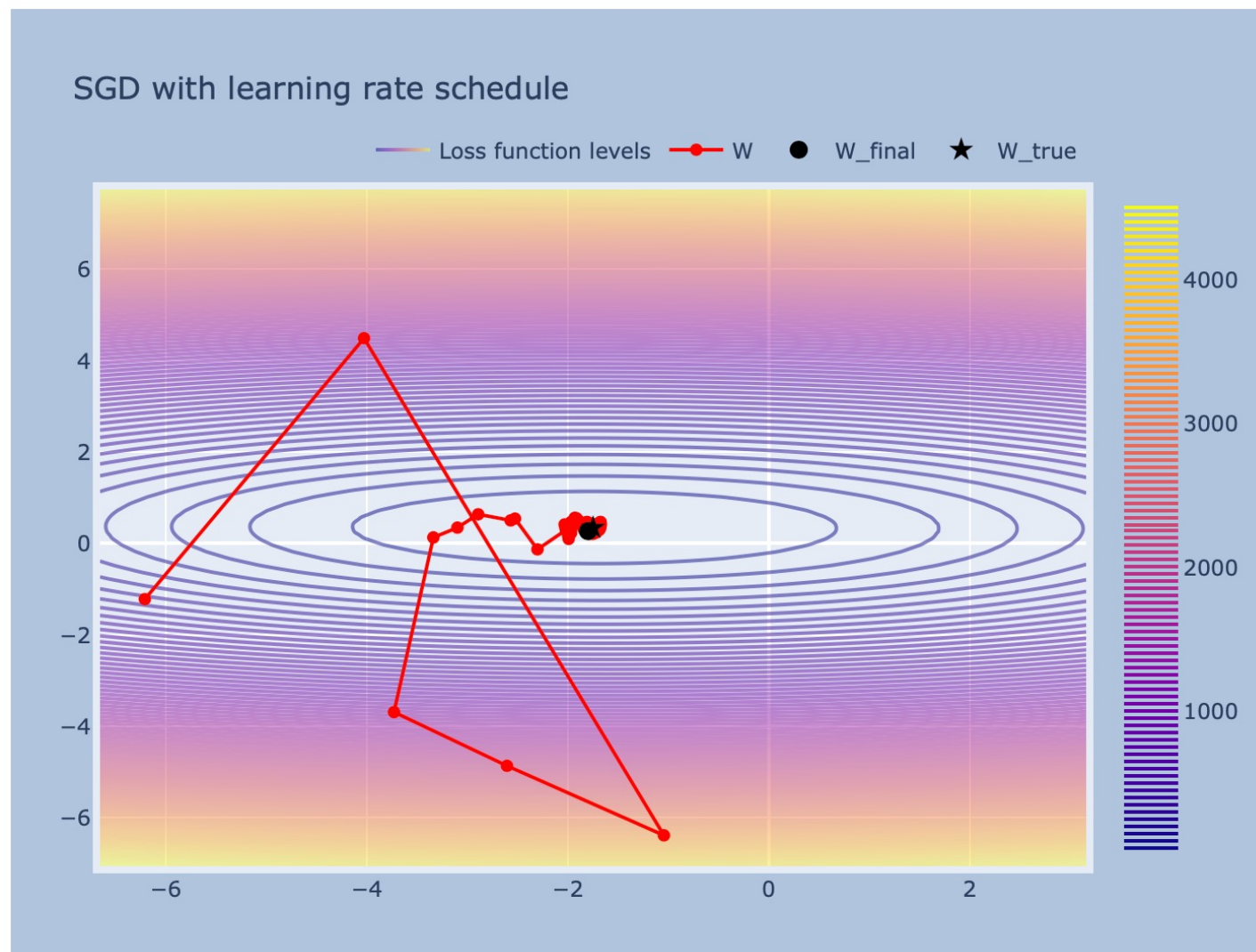
1. Начальное приближение: w^0
2. Повторять, каждый раз выбирая m случайных объектов $i_1, \dots, i_m$:

$$w^t = w^{t-1} - \eta_t \frac{1}{m} \sum_{j=1}^m \nabla L(y_{t,j}, a(x_{t,j}))$$

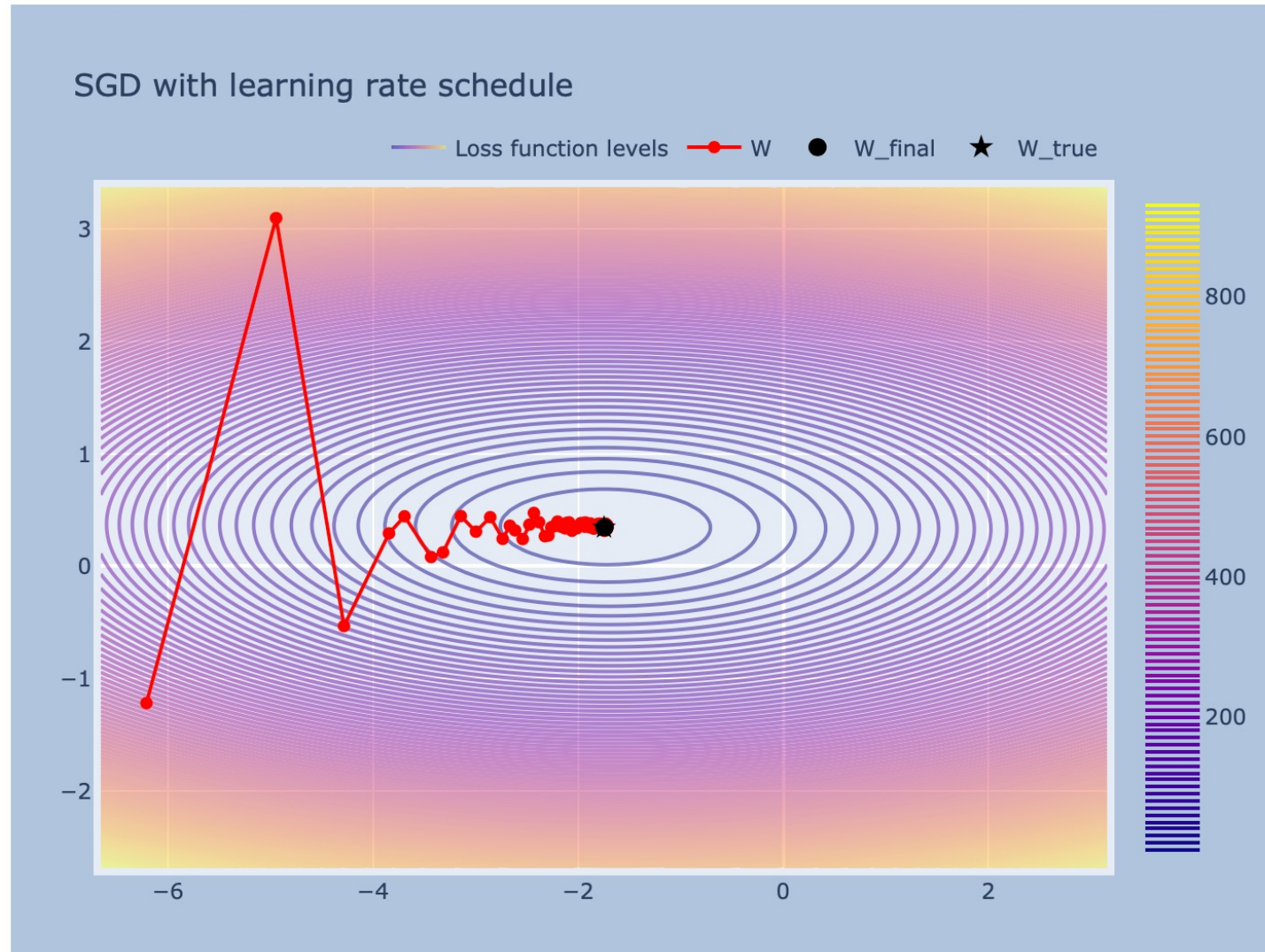
3. Останавливаемся, если ошибка на валидационной выборке перестаёт убывать

$x_{t,j}$ — объект номер j из батча, сформированного на шаге t

Батч размера 1



Батч размера 10



Batch size

- Размер батча — от десятков до тысяч
 - при обучении больших языковых моделей — миллионы токенов за шаг
- Степень двойки — скорее традиция: на современных GPU разница обычно незаметна
- Шум в оценке градиента падает как $\frac{1}{\sqrt{B}}$
 - чтобы уменьшить его вдвое, нужен батч в 4 раза больше
- Вычислительно почти так же эффективен, как шаг по градиенту одного объекта — за счёт векторизации

Batch size

The collected experimental results for the CIFAR-10, CIFAR-100 and ImageNet datasets show that increasing the mini-batch size progressively reduces the range of learning rates that provide stable convergence and acceptable test performance. On the other hand, small mini-batch sizes provide more up-to-date gradient calculations, which yields more stable and reliable training. The best performance has been consistently obtained for mini-batch sizes between $m = 2$ and $m = 32$, which contrasts with recent work advocating the use of mini-batch sizes in the thousands.

Batch size



Yann LeCun

April 27, 2018 · 🌐



Training with large minibatches is bad for your health.

More importantly, it's bad for your test error.

Friends dont let friends use minibatches larger than 32.

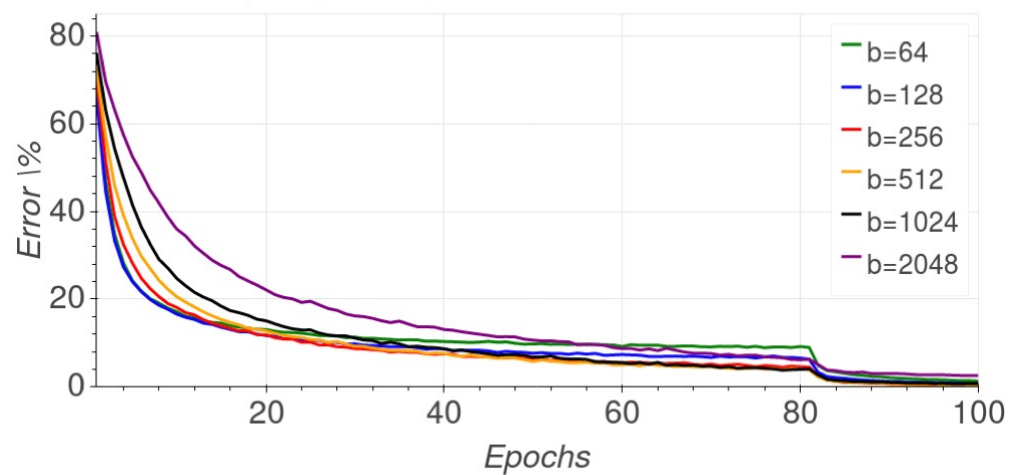
Let's face it: the **only** people have switched to minibatch sizes larger than one since 2012 is because GPUs are inefficient for batch sizes smaller than 32. That's a terrible reason. It just means our hardware sucks.

What's worse is that the easiest way to parallelize training is to make the minibatch even larger and distribute it across multiple GPUs and multiple nodes.

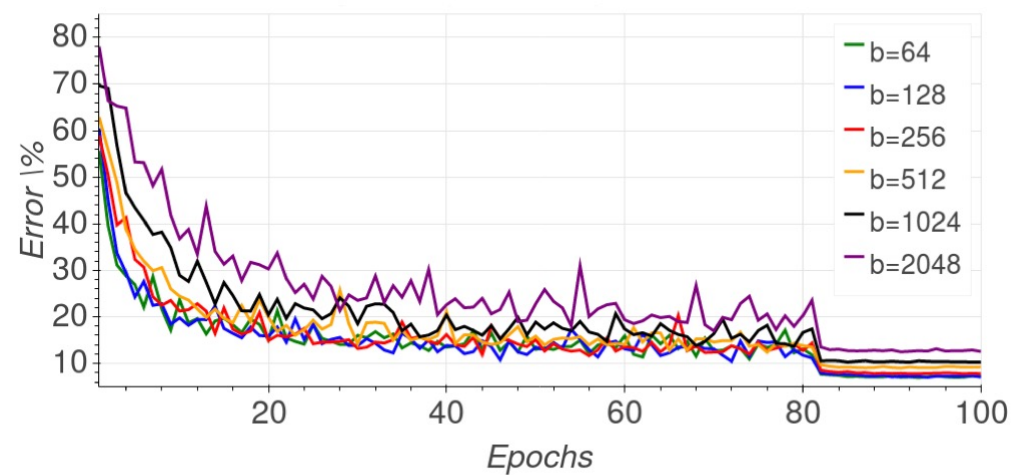
Minibatch sizes over 1024 aren't just bad for your health. They cause brain tumors.

They learn quickly, but the wrong thing.

Batch size

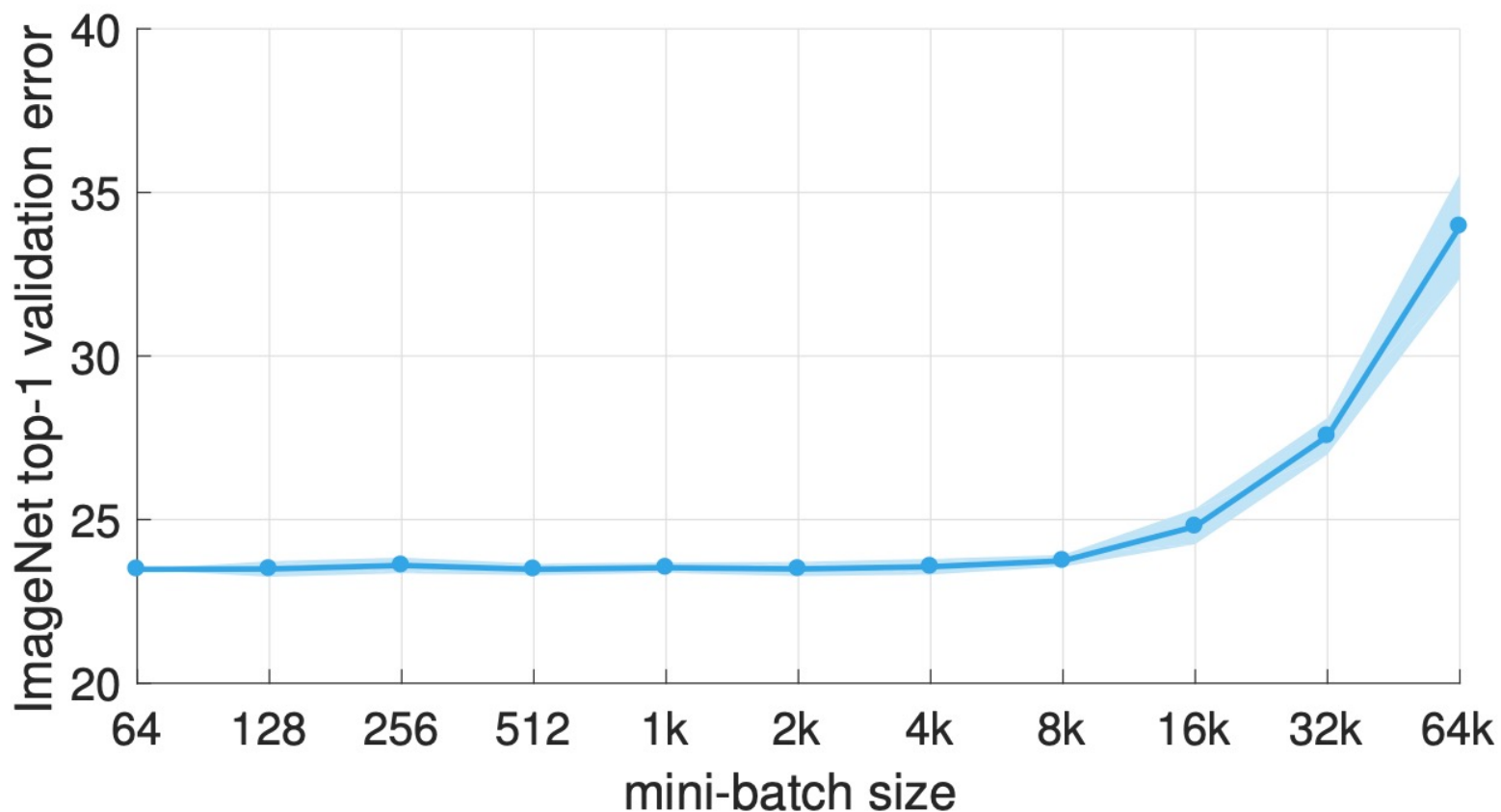


(a) Training error



(b) Validation error

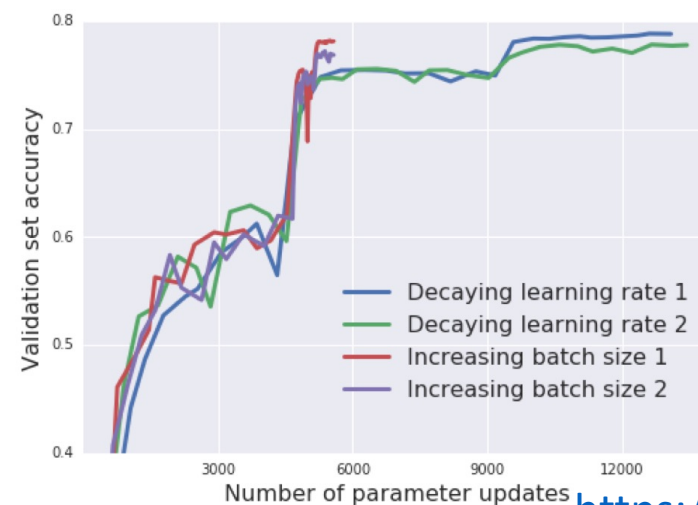
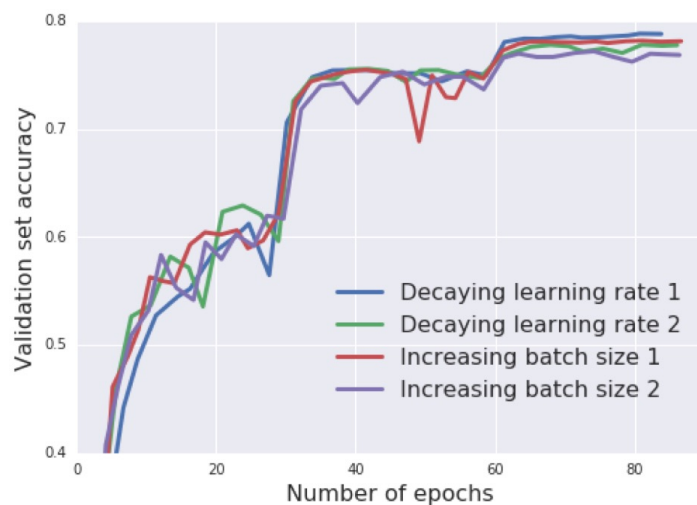
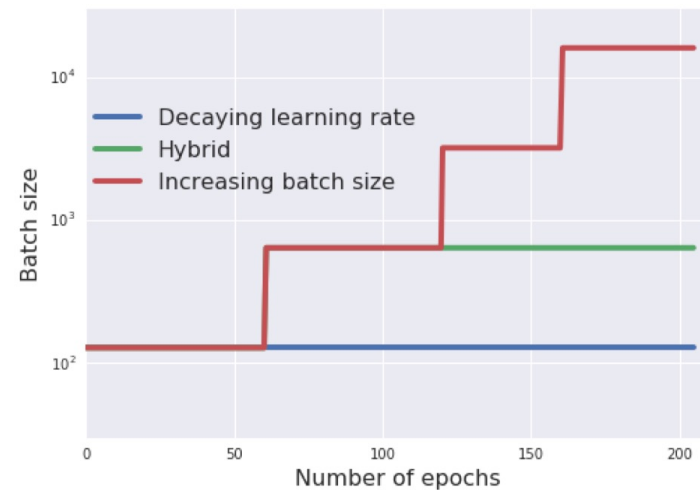
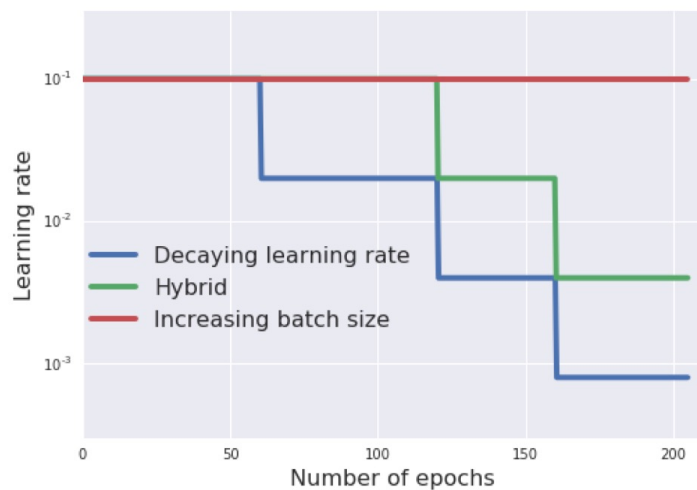
Надо грамотно подбирать формулу для длины шага!



Как подбирать длину шага для большого батча

- Linear scaling rule: увеличили батч в k раз $\rightarrow$ длину шага в k раз
- Warmup: первые несколько эпох длину шага линейно наращивают от маленькой до целевой
 - иначе большие шаги в начале обучения, пока веса случайные, разваливают модель
- Результат: обучаем ResNet-50 на ImageNet с батчем 8192 за 1 час на 256 GPU (без потери качества)
- Сейчас warmup используют почти везде, особенно при обучении трансформеров

Batch size



Критический размер батча

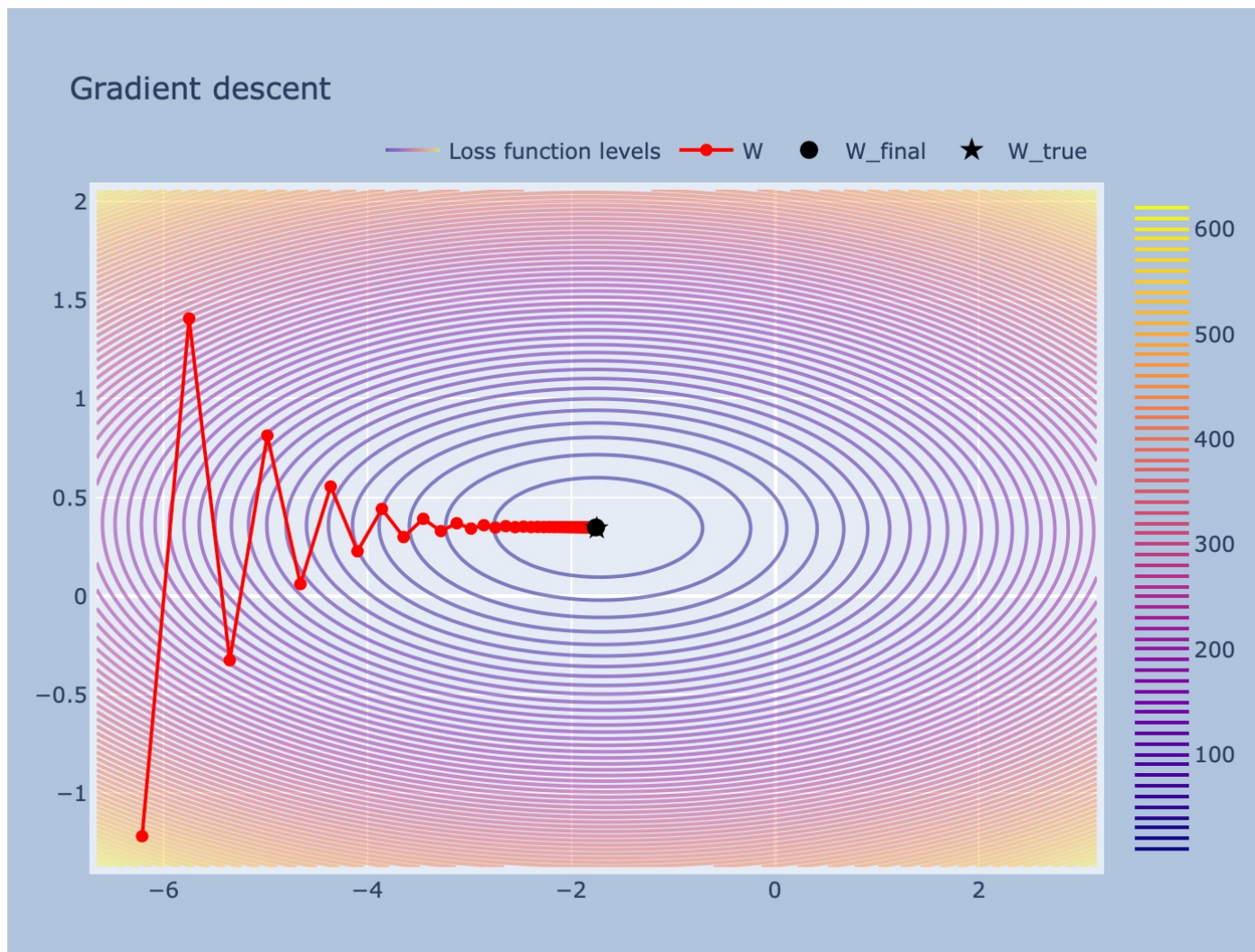
- До некоторого размера V_{crit} : увеличили батч вдвое → шагов нужно почти вдвое меньше (при правильно подобранной длине шага)
- Выше V_{crit} шагов почти не становится меньше — вычисления тратятся впустую
- V_{crit} сильно зависит от задачи: от ~20 до более чем 10 млн; для ImageNet — десятки тысяч
- V_{crit} растёт по ходу обучения → батч имеет смысл увеличивать постепенно (так обучали GPT-3)

Batch size

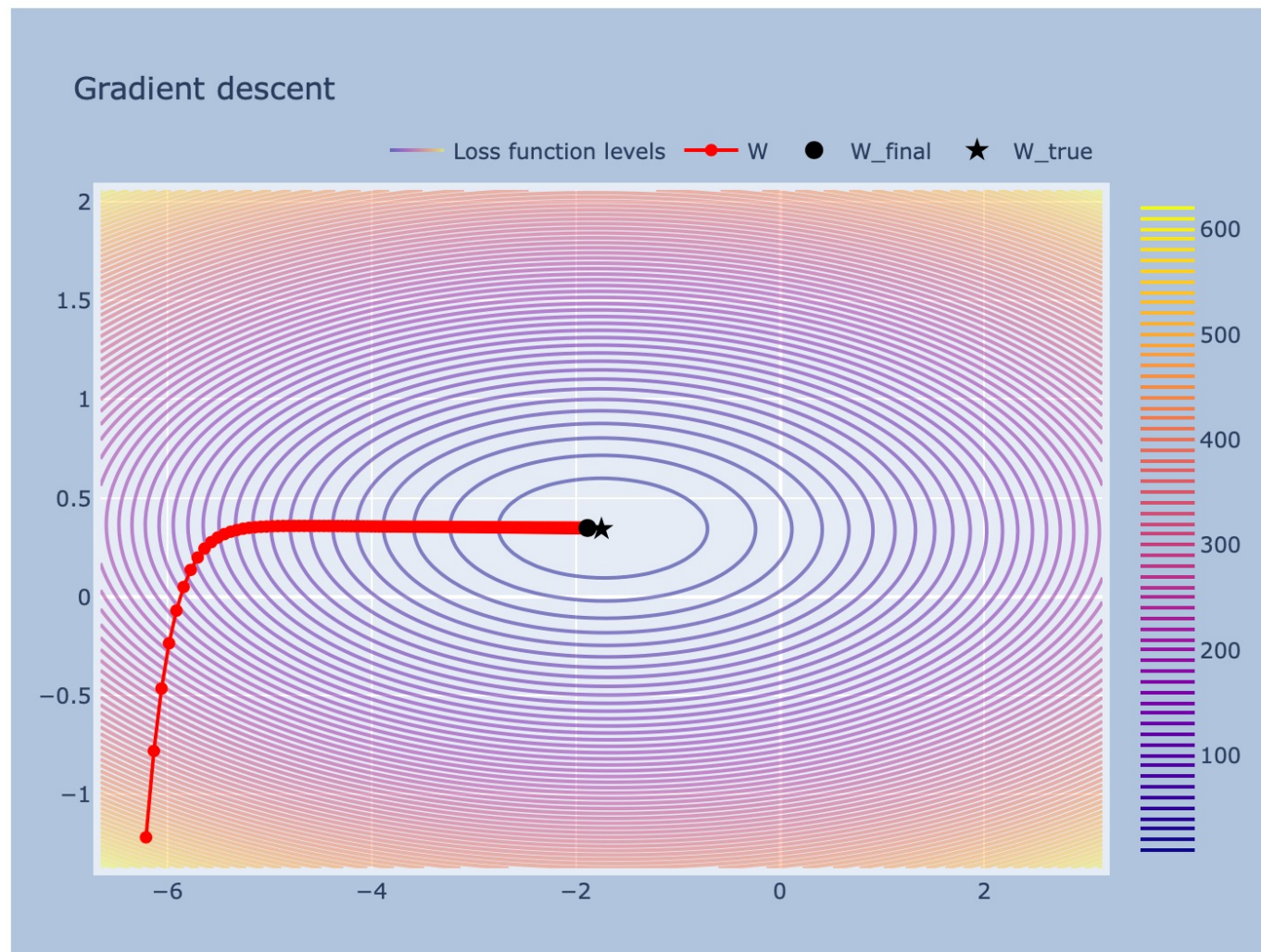
- От большого размера батча может быть польза
- Для этого надо грамотно подбирать длину шага и/или постепенно увеличить размер батча
- Польза от роста батча ограничена критическим размером

Модификации градиентного спуска

Проблемы



Проблемы



Проблемы

- Если у функции «вытянуты» линии уровня, то градиентный спуск требует аккуратного выбора длины шага и будет долго сходиться

Momentum

$$h_t = \alpha h_{t-1} + \eta_t \nabla Q(w^{t-1})$$

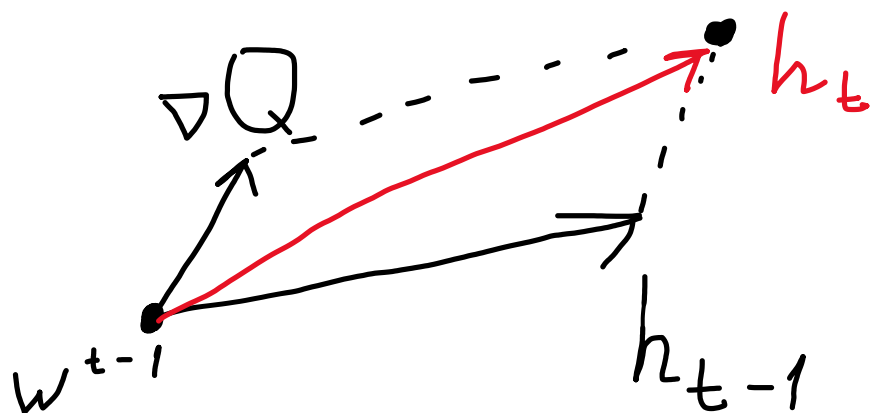
$$w^t = w^{t-1} - h_t$$

- h_t — «инерция», усреднённое направление движения
- α — параметр затухания
- Как будто шарик, который катится в сторону минимума, очень тяжёлый

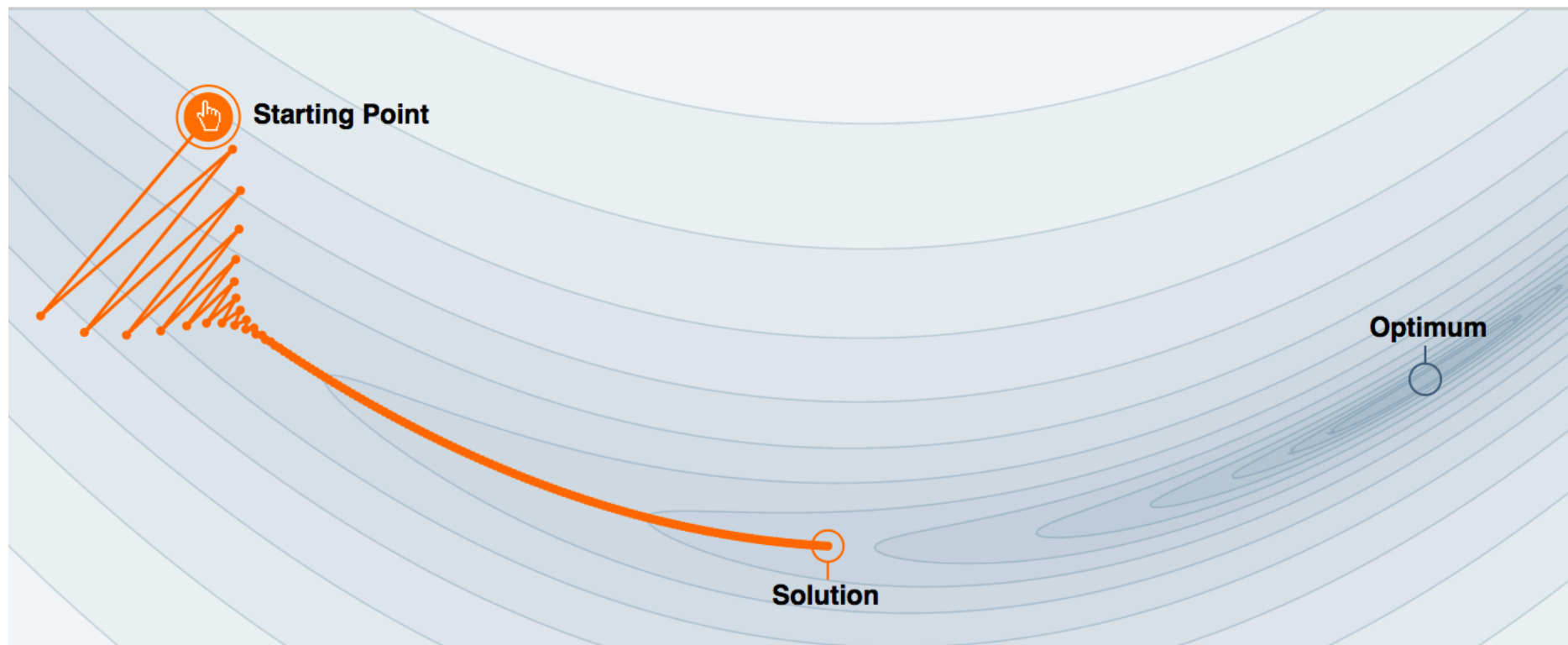
Momentum

$$h_t = \alpha h_{t-1} + \eta_t \nabla Q(w^{t-1})$$

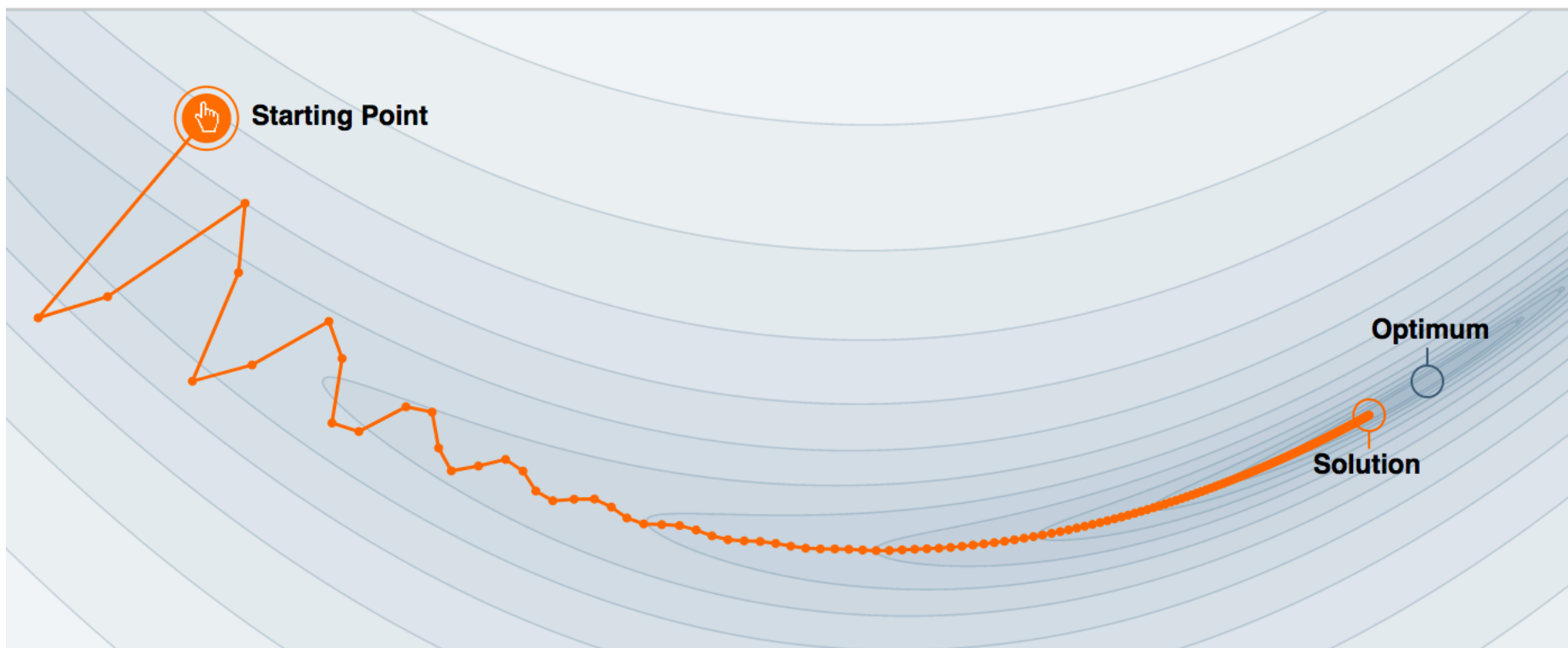
$$w^t = w^{t-1} - h_t$$



Без инерции



С инерцией



Nesterov Momentum

$$h_t = \alpha h_{t-1} + \eta_t \nabla Q(w^{t-1} - \alpha h_{t-1})$$

$$w^t = w^{t-1} - h_t$$

- $w^{t-1} - \alpha h_{t-1}$ — неплохая оценка того, куда мы попадём на следующем шаге

